

COMMIT LOCALLY, EXIT GLOBALLY: COORDINATING ADAPTIVE SAMPLING AND EARLY EXIT IN DIFFUSION LANGUAGE MODELS

Anonymous authors

Paper under double-blind review

ABSTRACT

Diffusion language models expose a provisional full-sequence prediction at every denoising step, and on many tasks the answer stabilizes well before decoding finishes. This creates two opportunities to reduce computation: leaving a block early and terminating the entire sequence early. These decisions operate at different scopes. Block acceleration is local, whereas sequence termination is global, and the two are not independent because local commitments can alter the evidence observed by a global exit rule. Existing approaches largely optimize these two control points separately, without accounting for how local commitments can influence the evidence used for global termination. In this paper, we present C^4 , a training-free framework that jointly coordinates local commitment and global termination. **Confidence-Verified Early Exit** (CVEE) decides *when* decoding may stop by re-extracting the current answer at every step and requiring both confidence and temporal stability. **Commit-Core-Then-Confirm** (CCTC) decides *which token positions* may commit within a block while preserving parallel prediction. It imposes an autoregressive-style freezing order by committing a boundary-anchored core first and confirming deferred positions one step later. For format-delimited answers, this design accelerates even the answer block while keeping global termination tied to the evolving answer trajectory. Across 12 zero-shot tasks with LLaDA and Dream, one fixed numerical configuration removes 64–95% of decoding steps and yields measured end-to-end speedups of 2.6×–18.6× while maintaining accuracy comparable to full decoding. CVEE provides most of the gains on short-answer tasks, whereas CCTC provides most of the savings for long reasoning and code generation, showing that global termination and local commitment offer complementary acceleration.

1 INTRODUCTION

Diffusion language models (DLMs) generate text by iteratively denoising a fully masked sequence rather than extending a prefix left to right (Nie et al., 2025; Ye et al., 2025), so each step predicts masked token positions in parallel with bidirectional context. This decoding process exposes two kinds of training-free slack. Every denoising step yields a *provisional* full-sequence prediction, and on many tasks the answer reaches its full-decoding value well before the final step. Practical decoders also run block by block under a semi-autoregressive schedule, so computation can be saved at two scopes: by leaving a block before its local budget is exhausted, or by terminating the entire sequence before all blocks finish.

These two decisions should not be treated as the same problem. Block acceleration is **local and bounded**: decoding continues after positions are committed. Sequence termination is **global and final**: once it fires, the remaining sequence, including the graded answer, can no longer be revised. The decision criterion should therefore match the scope of the action. Accelerating a block asks which positions are ready to commit, whereas stopping the sequence asks whether the *answer* itself has stabilized. Existing methods typically optimize only one control point. *Adaptive-sampling* methods such as *Fast-dLLM* (Wu et al., 2026), *SlowFast Sampling* (Wei et al., 2026), and *KLASS* (Kim et al., 2025) accelerate commitment while decoding continues, whereas generation-time *early-exit* methods such as *Prophet* (Li et al., 2026) and *Sched* (Mohamed et al., 2026b) decide when the sequence may stop using fixed-region confidence statistics or schedule-dependent rules rather than the answer itself. *WINO* (Hong et al., 2026) also acts on the sampling axis, making commits revocable through re-masking.

Simply combining the two is insufficient. First, neither decoding progress nor aggregate token-level margins reliably indicate answer convergence. In our measurements, the point by which 90% of answers have stabilized ranges from roughly 4% of decoding on short-answer tasks to 96% on long-reasoning tasks. A global rule tied to progress or aggregate margins can therefore stop too late in one regime and too early in another. Second, the two control points are coupled. The answer block is often also the block a sampler most wants to accelerate. If a local rule commits there aggressively, the answer may appear stable because the sampler froze it rather than because the model finished settling. Thus **local commitment can make the answer appear stable before it has actually converged**.

We therefore co-design the two decisions rather than compose an exit heuristic with an independent sampler. *Confidence-Verified Early Exit (CVEE)* decides *when* the sequence may stop by re-extracting the answer at every step and requiring both high confidence and temporal stability. *Commit-Core-Then-Confirm (CCTC)* decides *where* additional confidence-triggered commitments may occur within a block. CCTC preserves parallel diffusion prediction while imposing an autoregressive-style commitment order on these additional commits: a boundary-anchored core of ready positions may commit immediately, while threshold-ready positions beyond the core are deferred until their predictions are confirmed one step later.

The two mechanisms target complementary sources of redundant computation. CVEE provides most of the savings when the answer stabilizes early, as is common on short-answer tasks, whereas CCTC reduces computation within blocks that still need to execute on long-reasoning and code tasks. This coordination allows the same fixed numerical configuration to operate across all three regimes.

- We characterize answer stabilization in diffusion language models and show that its timing varies sharply across task regimes: short-answer tasks often stabilize early, whereas long-reasoning tasks may remain unsettled until near the decoding horizon. This variation limits stopping rules based only on decoding progress or aggregate confidence.
- We propose C^4 , which jointly coordinates local commitment and global sequence termination. CVEE tracks answer-level convergence to decide *when* to stop, while CCTC preserves parallel prediction but imposes an autoregressive-style order on additional local commitments, deciding *where* confident predictions may be safely frozen.
- We evaluate C^4 on 12 zero-shot tasks across two diffusion language model families using one fixed numerical configuration. It removes 64–95% of decoding steps and achieves measured end-to-end speedups of 2.6×–18.6× while maintaining accuracy comparable to full decoding.

2 RELATED WORKS

Diffusion language models. The diffusion framework traces to Sohl-Dickstein et al. (2015), with extensions to discrete data by Hooeboom et al. (2021); Austin et al. (2021a); Campbell et al. (2022). Lou et al. (2024) and the masked-diffusion line (Shi et al., 2024; Sahoo et al., 2024; Ou et al., 2025; Zheng et al., 2025) establish the parameterizations underlying modern DLMs. LLaDA (Nie et al., 2025) and Dream (Ye et al., 2025), the open-weight models we evaluate, expose revisable predictions at masked positions and admit flexible inference budgets. C^4 targets this inference-time budget under block-wise decoding by separating token-level commitment from answer-level termination.

Early exit and sequence termination. Early exit has been studied at the layer, reasoning-step, and token levels. Gu et al. (2026) apply it to *depth* via hidden-state stabilization. Among autoregressive CoT methods, S-GRPO (Dai et al., 2025), DEER (Yang et al., 2026), CORE (Zhai et al., 2026), and BMC (Ruan et al., 2026) trigger early stopping through decaying reward, transition confidence, brittle-token revision, and geometric reconstruction, respectively. For DLMs, Prophet (Li et al., 2026) and SchED (Mohamed et al., 2026b) stop using aggregate confidence signals over fixed monitoring regions or schedule-dependent thresholds rather than directly tracking the current answer. C^4 instead conditions global termination on confidence and stability of a dynamically extracted candidate span.

Adaptive sampling for DLMs. Adaptive sampling controls which token positions commit while decoding continues. Fast-dLLM (Wu et al., 2026), SlowFast Sampling (Wei et al., 2026), and KLASS (Kim et al., 2025) commit using token-level statistics, while concurrent work refines related signals: LESS (Mohamed et al., 2026a) combines confidence with stability, STDec (Chen et al., 2026) adapts thresholds over spatial neighborhoods, TACG (Wang et al., 2026) uses EMA-logit trajectories, Swordsman (Zhang et al., 2026) re-partitions blocks along entropy shifts, R^2 -dLLM (Du et al., 2026)

exploits spatio-temporal redundancy with fine-tuning, and Kim et al. (2026) identify proximity bias in the denoising order. WINO (Hong et al., 2026) is the closest relative to CCTC: it commits broadly and later re-masks disputed positions, whereas CCTC constrains the commit structure beforehand and never revokes. These methods accelerate commitment but do not decide whether the sequence itself is finished. Moreover, simply adding an exit rule does not remove the coupling: commitments inside the answer block can alter the stability observed by termination.

3 BACKGROUND AND PROBLEM SETUP

DLM generation process. A DLM generates a sequence x_0 of length L by reversing a discrete corruption process that progressively masks a clean sequence over forward time $u \in [0, 1]$,

$$q(x_u | x_0) = \prod_{i=1}^L q(x_u^i | x_0^i), \quad (1)$$

$$q(x_u^i | x_0^i) = \begin{cases} x_0^i, & \text{w.p. } 1 - u, \quad (\text{a) token kept} \\ [\text{MASK}], & \text{w.p. } u, \quad (\text{b) token masked} \end{cases} \quad (2)$$

so x_1 is fully masked and x_0 is the original sequence. The model $p_\theta(x_0 | x_u)$ learns to recover x_0 from masked inputs; generation reverses this process over T discrete *decoding steps* $t \in \{1, \dots, T\}$, distinct from forward time u . Each step predicts every masked position, after which a *remasking* rule commits a subset and leaves the rest masked. Since T is chosen at inference time, both the number of executed steps and the positions committed at each step are inference-time design choices.

Semi-autoregressive decoding. In practice, Eqs. 1 and 2 are not reversed over the entire sequence at once. The generation region is partitioned into N contiguous blocks $B_0, \dots, B_{N-1}$ and decoded left to right, with each block allocated $S = T/N$ steps. Diffusion therefore occurs within a block and autoregression across blocks. This is the schedule used by LLaDA-8B-Instruct (Nie et al., 2025) and Dream-7B-Instruct (Ye et al., 2025), and it creates the two control points we modify: when a block may advance and when the sequence may stop. Within each block, low-confidence remasking commits a fixed quota of the currently masked positions with highest confidence at every step. The quota size is schedule-determined rather than confidence-adaptive, ensuring that the block is fully committed by step S .

Early-termination gates. Prophet (Li et al., 2026) and SchED (Mohamed et al., 2026b) modify the second control point, deciding when the entire sequence may stop; adaptive samplers instead change how quickly positions commit while decoding continues. Prophet averages the top-1/top-2 logit gap over a monitored region $\mathcal{R}$,

$$\bar{g}_t = \frac{1}{|\mathcal{R}|} \sum_{i \in \mathcal{R}} \left(\ell_i^{(t),1} - \ell_i^{(t),2} \right), \quad (3)$$

where $\ell_i^{(t),1}$ and $\ell_i^{(t),2}$ are the two largest logits at position i and step t . We use a fixed task-format-specific definition of $\mathcal{R}$ in our evaluation, as detailed in Appendix A. Prophet fills all remaining masked positions once $\bar{g}_t$ crosses its published progress-dependent three-stage threshold; SchED replaces this with a progress-dependent decay schedule and a stability guard. Both reduce position-level signals to a single sequence-wide stopping trigger.

4 LOCAL COMMITMENT AND GLOBAL EXIT

Token-level statistics are useful for deciding *where* to commit, but are not sufficient to decide *when* the whole sequence should stop. Accordingly, CCTC controls local commitment and CVEE controls global termination. Figure 1 shows how the two sources of savings combine.

4.1 WHY PROGRESS-BASED GLOBAL STOPPING FAILS

Decoding progress is not the same as answer convergence. A simple global exit rule would stop after a fixed fraction of the decoding budget, implicitly assuming that different tasks stabilize at similar points. To test this assumption, we measure when a trajectory’s extracted answer stops changing. We take full LLaDA-8B-Instruct decodes drawn equally from MMLU, GSM8K, and MATH. Let $a_{j,r}$ denote the extracted candidate from trajectory j at normalized decoding progress r , and let a_j^{full} be its full-decoding answer. We define

$$s_j = \min\{s : a_{j,r} = a_j^{\text{full}} \forall r \geq s\}$$

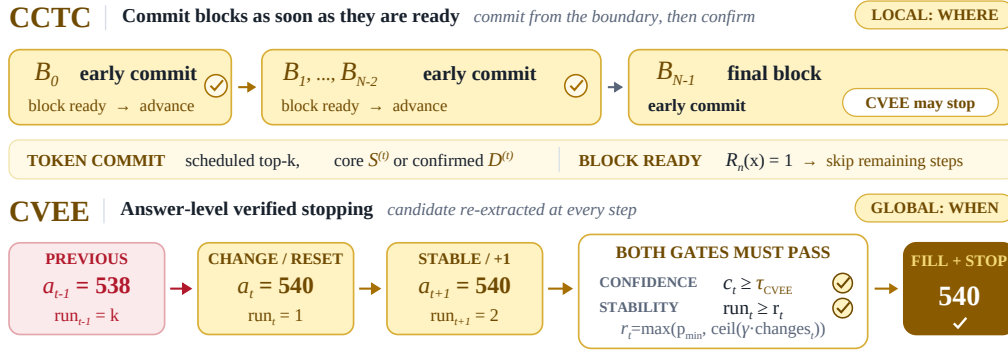


Figure 1: C^4 commits locally and exits globally. CCTC accelerates every block by committing a boundary-anchored core and confirming deferred positions one step later, including the final block; CVEE separately decides whether the sequence may be filled and stopped from answer-level evidence. No block-local readiness signal alone triggers global termination.

as trajectory j 's *stabilization progress*, and s_p as the progress by which a fraction p of trajectories have reached their own s_j . The criterion is intentionally strict: after s_j , the candidate never changes again.

A single progress threshold does not span the observed regimes. Across these trajectories, $s_{0.9}$ ranges from 0.04 on short-answer tasks to 0.96 on long-reasoning ones. A threshold early enough for MMLU would interrupt unfinished reasoning, whereas one late enough for MATH would waste most of the available slack elsewhere. The same issue applies to aggregate confidence: the monitored region can appear confident even while the extracted candidate is still changing. A global gate should therefore track the answer trajectory rather than decoding progress alone.

4.2 CONFIDENCE-VERIFIED EARLY EXIT: DECIDING WHEN TO STOP

Neither confidence nor stability alone establishes convergence. CVEE therefore tests both on the extracted candidate itself. At step t , the *candidate* is the normalized answer returned by a deterministic parser from the current prediction $\hat{x}^{(t)}$; the parser relocates the span as decoding proceeds. This requires a format-delimited answer unit that can be extracted consistently; outputs with no such unit fall outside CVEE's termination rule. Stability alone can arise from scheduled commits, while high confidence alone has no temporal information and cannot distinguish a settled answer from a transient prediction. Related early-exit methods in autoregressive reasoning face the same issue (Yang et al., 2026; Zhai et al., 2026; Ruan et al., 2026).

Confidence and candidate stability must hold jointly. Let a_t be the extracted candidate; run_t the number of successive same-value extractions (paused when no candidate exists); $changes_t$ the number of candidate flips; and $c_t \in [0, 1]$ the mean token confidence over the extracted span. CVEE fires at the first step t satisfying

$$c_t \geq \tau_{CVEE} \quad \text{(a) confidence gate}$$

$$run_t \geq \max(p_{\min}, \lceil \gamma \cdot changes_t \rceil) \quad \text{(b) stability gate.} \quad (4)$$

Termination requires both conditions. $p_{\min}$ prevents immediate termination, while γ requires additional stable observations after each flip. The three parameters (τ_{CVEE} , γ , $p_{\min}$) are calibrated once on a shared LLaDA pool and frozen thereafter. The numerical gate itself is schedule-independent, although candidate availability is format-dependent. The fixed parser searches a task-format region, and the counters pause while no candidate can yet be extracted. Once a candidate is available, the trigger depends on its confidence and trajectory rather than on decoding progress, so different stopping times arise from the candidate trajectories themselves rather than from a task router (Lee et al., 2026).

4.3 COMMIT-CORE-THEN-CONFIRM: WHERE TO ACCELERATE, AND HOW MANY TO COMMIT

Threshold commits can detach tokens from unresolved context. Let $E^{(t)}$ be the positions whose confidence exceeds τ_{CCTC} at step t . If a later token is committed while earlier positions remain masked, that token is frozen before its left context is resolved and is never re-decoded. CCTC therefore keeps masked-position prediction parallel but constrains the order of additional confidence-triggered

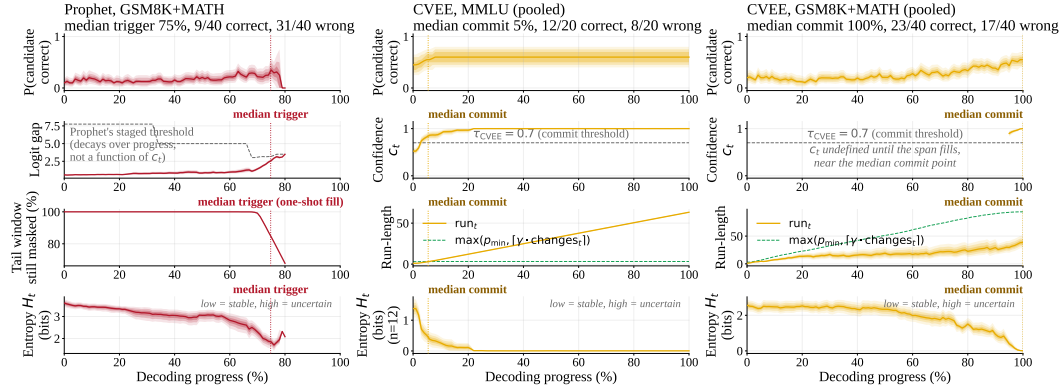


Figure 2: CVEE and Prophet trigger timing versus answer stabilization. Prophet can trigger before the answer stabilizes on GSM8K and MATH, whereas CVEE fires early on MMLU and waits on long-reasoning trajectories until its tested candidate settles. Predictive entropy at the trigger points shows the same contrast, indicating that the difference lies in *what* the gates test rather than only in threshold strictness. Examples shown here are excluded from CVEE’s calibration.

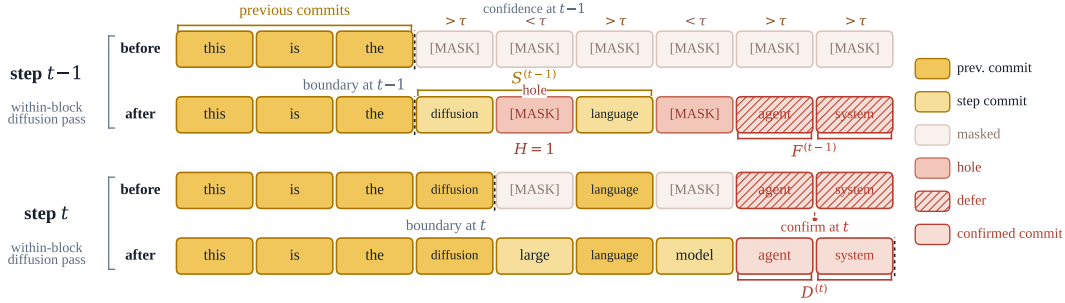


Figure 3: CCTC stays block-parallel under an autoregressive freezing order. At step t , the core $S^{(t)}$ commits from the current boundary and may cross at most H low-confidence holes; later ready positions remain masked and are deferred. At the same step, $D^{(t)}$ confirms positions deferred at $t-1$ after additional left context has been resolved.

commitments. Boundary-near positions commit first, while later ready positions are deferred for confirmation.

Autoregressive freezing within a block. CCTC borrows the left-to-right freezing order of an autoregressive decoder without serializing prediction. Its *core* is the boundary-anchored subset of $E^{(t)}$ that crosses at most H unresolved low-confidence holes:

$$h_i^{(t)} = |\{j \in B_n : j < i, x_j^{(t)} = [\text{MASK}], j \notin E^{(t)}\}|, \quad S^{(t)} = \{i \in E^{(t)} : h_i^{(t)} \leq H\}. \quad (5)$$

Here $h_i^{(t)}$ counts unresolved holes before i . Already committed positions do not count, so the core remains anchored at the resolved boundary. Positions in $E^{(t)} \setminus S^{(t)}$ are **deferred**: they stay masked for one step while their predictions are recorded. Only the freezing order is autoregressive; prediction remains parallel.

Confirmation without an extra forward pass. Let $F^{(t-1)} = E^{(t-1)} \setminus (S^{(t-1)} \cup D^{(t-1)})$ be the positions deferred at $t-1$, and let $\tilde{x}_i^{(t-1)}$ be the prediction recorded for each of them. The confirmed set at step t is

$$D^{(t)} = \{i \in F^{(t-1)} \cap E^{(t)} : x_i^{(t)} = [\text{MASK}], \hat{x}_i^{(t)} = \tilde{x}_i^{(t-1)}\}, \quad (6)$$

so a deferred position commits only if it still clears τ_{CCTC} and still predicts the recorded token once more left context has been resolved. A deferred position that fails confirmation stays masked. If it remains above the threshold without entering the core, it is deferred again with its new prediction, and otherwise it leaves the deferral buffer. Deferrals do not carry across block boundaries. Unlike WINO (Hong et al., 2026), CCTC never revokes and introduces no separate verification pass. Let $\mathcal{A}$ be the accelerated blocks and $\mathcal{K}^{(t)}$ the reference decoder’s scheduled top- k selection. Every masked

$i \in B_n, n \in \mathcal{A}$, updates as

$$x_i^{(t+1)} = \begin{cases} \hat{x}_i^{(t)}, i \in S^{(t)} & \text{(a) core} \\ \hat{x}_i^{(t)}, i \in D^{(t)} & \text{(b) confirmed deferral} \\ \hat{x}_i^{(t)}, i \in \mathcal{K}^{(t)} & \text{(c) scheduled top-}k \\ x_i^{(t)}, & \text{otherwise (d) stays masked.} \end{cases} \quad (7)$$

CCTC constrains only the additional threshold-driven commits in branches (a) and (b), while the reference decoder’s scheduled top- k branch (c) is retained unchanged to guarantee progress. As $H \rightarrow \infty$, $S^{(t)} = E^{(t)}$ and structural deferral vanishes. Once all positions in a block have committed,

$$R_n(x) = \bigwedge_{i \in B_n} [x_i \neq [\text{MASK}]] \quad (8)$$

holds and the remaining block steps are skipped.

Accelerating even the answer block. The final design choice is which blocks CCTC accelerates. Restricting $\mathcal{A} = \{0, \dots, N-2\}$ would avoid additional commits in the final block; we instead use $\mathcal{A} = \{0, \dots, N-1\}$. CCTC still constrains extra value-based commits there through the boundary-anchored core and one-step confirmation, while CVEE bases global termination on the extracted answer trajectory rather than block readiness. Appendix C directly tests whether these commits manufacture CVEE’s evidence for short-span answers and also identifies the whole-program case where that separation no longer holds.

5 EXPERIMENTS

Datasets and setup. We evaluate LLaDA-8B-Instruct (Nie et al., 2025) and Dream-7B-Instruct (Ye et al., 2025) with bf16, greedy decoding, and zero-shot prompting on a single RTX 4090. The 12 tasks comprise five *short-answer* benchmarks (MMLU (Hendrycks et al., 2021a), ARC-Challenge (Clark et al., 2018), HellaSwag (Zellers et al., 2019), WinoGrande (Sakaguchi et al., 2021), and PIQA (Bisk et al., 2020)), five *long-reasoning* benchmarks (GSM8K (Cobbe et al., 2021), MATH (Hendrycks et al., 2021b), SVAMP (Patel et al., 2021), ASDiv (Miao et al., 2020), and GSM-Hard (Gao et al., 2023)), and two *code-generation* benchmarks (HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021b)), both graded by hidden unit tests.

Hyperparameters and calibration. C^4 uses five numerical hyperparameters throughout the paper. CVEE uses $\gamma=2.0$, $\tau_{\text{CVEE}}=0.7$, and $p_{\min}=3$; CCTC uses $\tau_{\text{CCTC}}=0.8$ and gap budget $H=1$. The three CVEE parameters are calibrated once on a shared 120-example LLaDA pool and then frozen across all 12 tasks and transferred unchanged to Dream. The two CCTC parameters are fixed *a priori* and are not calibrated per task or model. Because CCTC changes which decoding states are subsequently visited, it cannot be faithfully calibrated by offline replay without running the modified decoder. Peer methods use their published or released configurations. Appendix A reports the complete hyperparameter, extractor, and peer settings.

Evaluation metrics. Accuracy follows task-specific answer extraction, or execution grading for code. Step is the number of executed decoding steps, and Saving is $1 - \text{Step}/T_{\text{full}}$. Speedup is measured TPS relative to the corresponding full-decoding baseline. The 2.0-point tolerance in Table 1 is used only as a visual marker and is not used for calibration.

5.1 MAIN RESULTS AND ANALYSIS

Table 1 reports results on all 12 tasks and both backbones under a single frozen numerical configuration. Figure 4 compares each exit trigger with the point at which that example’s answer stabilizes. The two mechanisms exploit different sources of slack: CVEE removes decoding after the answer has settled, whereas CCTC reduces work inside blocks that must still execute.

Short-answer tasks. CVEE dominates this regime. On LLaDA, C^4 needs only 4.8 and 3.9 steps on MMLU and ARC-C because the candidate typically stabilizes within the first block, leaving little within-block work for CCTC to remove. Margin-based exit rules can continue tracking a position-level statistic after the answer has stabilized, while adaptive samplers do not test answer convergence at all. CVEE therefore removes most of the remaining trajectory, producing the largest wall-clock gains in this regime.

Long-reasoning tasks. The pattern reverses because answers stabilize near the decoding horizon. CVEE consequently fires late or not at all, while CCTC provides most of the savings by accelerating blocks that still need to execute. Dream GSM8K illustrates this split: CVEE alone leaves the full

Task	Variant	LLaDA-8B-Instruct					Dream-7B-Instruct				
		Acc (%)	Avg. Step	Saving	TPS	Speedup	Acc (%)	Avg. Step	Saving	TPS	Speedup
General / short-answer tasks (single-token or short-span answers)											
MMLU (n=200, sequence length=64) (block=16, 4 blocks)	Baseline	64.0	64.0	0.0%	30.4	1.00x	71.5	64.0	0.0%	28.2	1.00x
	Prophet (Li et al., 2026)	62.5 (-1.5)	8.2 (-55.8)	87.1%	202.2	6.70x	71.5 (+0.0)	14.9 (-49.2)	76.8%	109.8	3.89x
	SchED (Mohamed et al., 2026b)	61.5 (-2.5)	31.0 (-33.0)	51.6%	53.3	1.76x	71.5 (+0.0)	13.0 (-53.0)	79.8%	178.7	6.33x
	SlowFast (Wei et al., 2026)	63.0 (-1.0)	17.9 (-46.1)	72.0%	130.4	4.28x	74.5 (+3.0)	9.6 (-54.4)	85.0%	196.9	6.97x
	KLASS (Kim et al., 2025)	62.5 (-1.5)	29.2 (-34.8)	54.4%	51.7	1.69x	72.5 (+1.0)	11.3 (-52.7)	82.3%	129.0	4.57x
	WINO (Hong et al., 2026)	61.0 (-3.0)	16.8 (-47.2)	73.8%	100.4	3.38x	-	-	-	-	-
C ⁴	64.0 (+0.0)	4.8 (-59.2)	92.5%	490.4	16.16x	70.0 (-1.5)	4.5 (-59.5)	93.0%	389.7	13.82x	
ARC-C (n=200, sequence length=64) (block=16, 4 blocks)	Baseline	86.5	64.0	0.0%	33.8	1.00x	87.5	64.0	0.0%	32.6	1.00x
	Prophet (Li et al., 2026)	86.0 (-0.5)	7.5 (-56.5)	88.3%	295.3	8.51x	86.0 (-1.5)	14.0 (-50.0)	78.2%	138.3	4.25x
	SchED (Mohamed et al., 2026b)	85.0 (-1.5)	41.1 (-22.9)	35.8%	57.5	1.65x	88.5 (+1.0)	11.0 (-53.0)	82.8%	215.1	6.61x
	SlowFast (Wei et al., 2026)	85.0 (-1.5)	23.7 (-40.3)	63.0%	107.3	3.08x	87.0 (-0.5)	10.4 (-53.6)	83.8%	219.4	6.75x
	KLASS (Kim et al., 2025)	87.0 (+0.5)	37.5 (-26.5)	41.5%	53.7	1.55x	88.5 (+1.0)	11.6 (-52.4)	81.9%	158.9	4.88x
	WINO (Hong et al., 2026)	84.0 (-2.5)	20.1 (-43.9)	68.6%	76.9	2.33x	-	-	-	-	-
C ⁴	86.5 (+0.0)	3.9 (-60.1)	94.0%	483.3	14.28x	88.5 (+1.0)	4.2 (-59.8)	93.5%	565.7	17.37x	
HellaSwag (n=200, sequence length=64) (block=16, 4 blocks)	Baseline	75.5	64.0	0.0%	20.8	1.00x	75.5	64.0	0.0%	22.1	1.00x
	Prophet (Li et al., 2026)	75.0 (-0.5)	6.0 (-58.0)	90.6%	223.4	10.74x	75.0 (-0.5)	16.2 (-47.8)	74.7%	82.0	3.72x
	SchED (Mohamed et al., 2026b)	75.5 (+0.0)	27.7 (-36.3)	56.7%	46.4	2.23x	75.0 (-0.5)	19.0 (-45.0)	70.3%	63.0	2.85x
	SlowFast (Wei et al., 2026)	77.0 (+1.5)	109.0 (-53.1)	83.0%	134.2	6.44x	72.5 (-3.0)	9.0 (-55.0)	85.9%	160.0	7.25x
	KLASS (Kim et al., 2025)	75.0 (-0.5)	19.4 (-44.6)	69.6%	56.5	2.72x	69.5 (-6.0)	11.3 (-52.7)	82.3%	94.0	4.25x
	WINO (Hong et al., 2026)	73.0 (-2.5)	12.9 (-51.1)	79.8%	79.0	3.83x	-	-	-	-	-
C ⁴	75.5 (+0.0)	3.7 (-60.3)	94.3%	348.6	16.72x	75.5 (+0.0)	4.4 (-59.6)	93.1%	297.8	13.46x	
WinoGrande (n=200, sequence length=64) (block=16, 4 blocks)	Baseline	75.5	64.0	0.0%	36.1	1.00x	71.5	64.0	0.0%	35.5	1.00x
	Prophet (Li et al., 2026)	76.0 (+0.5)	5.9 (-58.1)	90.8%	383.9	10.24x	69.5 (-2.0)	15.3 (-48.7)	76.1%	140.6	3.98x
	SchED (Mohamed et al., 2026b)	75.5 (+0.0)	5.8 (-58.2)	91.0%	447.1	11.77x	71.0 (-0.5)	2.1 (-61.9)	96.7%	1034.4	29.08x
	SlowFast (Wei et al., 2026)	77.5 (+2.0)	85.1 (-52.5)	86.7%	285.5	7.64x	71.0 (-0.5)	8.0 (-56.0)	87.5%	279.6	7.85x
	KLASS (Kim et al., 2025)	76.5 (+1.0)	13.3 (-50.7)	79.2%	160.0	4.21x	72.5 (+1.0)	7.6 (-56.4)	88.2%	236.8	6.68x
	WINO (Hong et al., 2026)	77.0 (+1.5)	10.1 (-53.9)	84.2%	176.1	5.00x	-	-	-	-	-
C ⁴	75.5 (+0.0)	3.4 (-60.6)	94.6%	672.8	18.66x	71.5 (+0.0)	4.2 (-59.8)	93.4%	561.4	15.83x	
PIQA (n=200, sequence length=64) (block=16, 4 blocks)	Baseline	81.5	64.0	0.0%	35.2	1.00x	87.5	64.0	0.0%	32.5	1.00x
	Prophet (Li et al., 2026)	82.0 (+0.5)	10.8 (-53.2)	83.1%	245.7	6.86x	87.5 (+0.0)	18.9 (-45.2)	70.5%	104.0	3.19x
	SchED (Mohamed et al., 2026b)	81.0 (-0.5)	24.5 (-39.5)	61.7%	96.6	2.81x	87.0 (-0.5)	17.1 (-46.9)	73.3%	129.1	3.97x
	SlowFast (Wei et al., 2026)	84.0 (+2.5)	100.0 (-54.0)	84.4%	205.2	5.95x	89.5 (+2.0)	9.1 (-55.0)	85.9%	209.2	6.42x
	KLASS (Kim et al., 2025)	81.5 (+0.0)	18.6 (-45.4)	70.9%	106.3	2.96x	91.5 (+0.0)	10.6 (-53.4)	83.4%	158.0	4.85x
	WINO (Hong et al., 2026)	81.0 (-0.5)	11.4 (-52.6)	82.2%	187.7	5.47x	-	-	-	-	-
C ⁴	82.0 (+0.5)	3.6 (-60.4)	94.4%	546.5	15.52x	88.0 (+0.5)	3.5 (-60.5)	94.5%	575.7	17.69x	
Long-reasoning tasks (multi-step CoT)											
GSM8K (n=100, sequence length=256) (block=32, 8 blocks)	Baseline	66.0	256.0	0.0%	19.0	1.00x	85.0	256.0	0.0%	21.2	1.00x
	Prophet (Li et al., 2026)	38.0 (-28.0)	185.0 (-71.0)	27.7%	22.6	1.19x	16.0 (-69.0)	76.1 (-179.9)	70.3%	67.0	3.16x
	SchED (Mohamed et al., 2026b)	41.0 (-17.0)	207.0 (-31.7)	12.4%	21.1	1.11x	18.0 (-67.0)	76.9 (-179.1)	70.3%	64.9	3.07x
	SlowFast (Wei et al., 2026)	42.0 (-25.0)	70.9 (-185.1)	72.3%	68.0	3.57x	20.0 (-65.0)	89.2 (-166.8)	65.1%	59.6	2.81x
	KLASS (Kim et al., 2025)	65.0 (-1.0)	105.5 (-150.5)	58.8%	34.5	1.82x	33.0 (-52.0)	96.5 (-159.5)	62.3%	37.2	1.75x
	WINO (Hong et al., 2026)	72.0 (+6.0)	49.2 (-206.8)	80.8%	76.6	4.07x	-	-	-	-	-
C ⁴	64.0 (-2.0)	62.5 (-193.5)	75.6%	67.5	3.55x	85.0 (+0.0)	54.5 (-201.5)	78.7%	102.1	4.82x	
MATH (n=100, sequence length=256) (block=32, 8 blocks)	Baseline	32.0	256.0	0.0%	17.8	1.00x	42.0	256.0	0.0%	19.4	1.00x
	Prophet (Li et al., 2026)	19.0 (-13.0)	192.7 (-63.3)	24.7%	21.4	1.20x	15.0 (-27.0)	164.9 (-91.1)	35.6%	26.7	1.38x
	SchED (Mohamed et al., 2026b)	26.0 (-6.0)	233.5 (-22.5)	16.8%	19.3	1.08x	33.0 (-40.0)	199.0 (-60.0)	22.9%	23.3	1.20x
	SlowFast (Wei et al., 2026)	24.0 (-8.0)	100.0 (-156.0)	60.9%	45.5	2.55x	9.0 (-33.0)	98.9 (-157.1)	61.4%	51.9	2.67x
	KLASS (Kim et al., 2025)	34.0 (+2.0)	133.9 (-122.1)	47.7%	26.1	1.46x	4.0 (-38.0)	98.7 (-157.3)	61.4%	40.3	2.08x
	WINO (Hong et al., 2026)	35.0 (+3.0)	81.7 (-174.3)	68.1%	42.8	2.42x	-	-	-	-	-
C ⁴	33.0 (+1.0)	92.5 (-163.5)	63.9%	46.7	2.61x	42.0 (+0.0)	84.6 (-171.5)	67.0%	53.5	2.76x	
SVAMP (n=100, sequence length=256) (block=32, 8 blocks)	Baseline	81.0	256.0	0.0%	19.3	1.00x	76.0	256.0	0.0%	21.4	1.00x
	Prophet (Li et al., 2026)	40.0 (-41.0)	160.1 (-96.0)	37.5%	27.5	1.42x	29.0 (-47.0)	48.8 (-207.2)	80.9%	76.1	3.57x
	SchED (Mohamed et al., 2026b)	83.0 (+2.0)	196.0 (-60.0)	21.9%	23.9	1.24x	42.0 (-56.0)	42.9 (-213.1)	35.8%	53.6	2.50x
	SlowFast (Wei et al., 2026)	46.0 (-35.0)	81.7 (-174.3)	68.1%	63.4	3.28x	67.0 (-9.0)	49.9 (-206.1)	80.5%	137.0	6.42x
	KLASS (Kim et al., 2025)	79.0 (-2.0)	96.4 (-159.6)	62.4%	39.9	2.07x	71.0 (-5.0)	58.1 (-197.9)	77.3%	71.0	3.33x
	WINO (Hong et al., 2026)	85.0 (+4.0)	45.6 (-210.4)	82.2%	81.0	4.24x	-	-	-	-	-
C ⁴	81.0 (+0.0)	58.8 (-197.2)	77.0%	82.8	4.28x	75.0 (-1.0)	42.8 (-213.2)	83.3%	113.0	5.29x	
ASDiv (n=100, sequence length=256) (block=32, 8 blocks)	Baseline	68.0	256.0	0.0%	19.4	1.00x	78.0	256.0	0.0%	21.4	1.00x
	Prophet (Li et al., 2026)	37.0 (-31.0)	163.7 (-92.3)	36.1%	29.5	1.52x	25.0 (-53.0)	54.8 (-201.3)	78.6%	122.5	5.75x
	SchED (Mohamed et al., 2026b)	60.0 (-8.0)	197.0 (-59.0)	23.0%	28.6	1.48x	17.0 (-61.0)	53.3 (-202.7)	79.2%	101.1	4.74x
	SlowFast (Wei et al., 2026)	34.0 (-34.0)	82.4 (-173.7)	67.8%	64.9	3.35x	52.0 (-26.0)	57.9 (-198.1)	77.4%	144.0	6.78x
	KLASS (Kim et al., 2025)	66.0 (-2.0)	100.8 (-155.2)	60.6%	40.7	2.10x	58.0 (-20.0)	67.2 (-188.8)	73.7%	65.4	3.07x
	WINO (Hong et al., 2026)	60.0 (-8.0)	48.2 (-207.8)	81.2%	84.5	4.41x	-	-	-	-	-
C ⁴	71.0 (+3.0)	60.9 (-195.1)	76.2%	86.9	4.48x	76.0 (-2.0)	45.8 (-210.2)	82.1%	135.1	6.32x	
GSM-Hard (n=100, sequence length=256) (block=32, 8 blocks)	Baseline	35.0	256.0	0.0%	18.8	1.00x	42.0	256.0	0.0%	20.6	1.00x
	Prophet (Li et al., 2026)	18.0 (-17.0)	194.1 (-61.9)	24.2%	21.9	1.16x	9.0 (-33.0)	103.9 (-152.1)	59.4%	45.7	2.22x
	SchED (Mohamed et al., 2026b)	31.0 (-4.0)	257.4 (-18.6)	7.3%	19.7	1.05x	10.0 (-32.0)	123.5 (-132.6)	51.8%	41.6	2.02x
	SlowFast (Wei et al., 2026)	21.0 (-14.0)	70.0 (-186.0)	72.6%	70.2	3.72x	11.0 (-31.0)	96.8 (-159.2)	62.2%	62.6	3.04x
	KLASS (Kim et al., 2025)	35.0 (+0.0)	109.7 (-146.3)	57.1%	34.0	1.81x	10.0 (-32.0)	111.7 (-144.3)	56.4%	35.1	1.71x
	WINO (Hong et al., 2026)	34.0 (-1.0)	51.2 (-204.8)	80.0%	77.3	4.15x	-	-	-	-	-
C ⁴	37.0 (+2.0)	64.4 (-191.6)	74.8%	67.6	3.59x	45.0 (+3.0)	64.9 (-191.1)	74.7%	82.2	3.98x	
Code generation (whole-program answer, execution-graded)											
HumanEval (n=164, sequence length=384) (block=32, 12 blocks)	Baseline	40.2	384.0	0.0%	13.8	1.00x	45.1	384.0	0.0%	14.5	1.00x
	Prophet (Li et al., 2026)	3.0 (-37.2)	124.0 (-260.0)	67.7%	30.9	2.24x	0.6 (-44.5)	29.3 (-354.7)	92.4%	144.2	9.93x
	SchED (Mohamed et al., 2026b)	12.8 (-27.4)	175.7 (-208.3)	54.3%	33.1	2.39x	1.8 (-43.3)	39.3 (-344.7)	89.8%	142.9	9.84x
	SlowFast (Wei et al., 2026)	19.5 (-20.7)	36.3 (-347.7)	90.5%	213.1	15.42x	23.2 (-22.0)	36.4 (-347.6)	90.5%	205.6	14.16x
	KLASS (Kim et al., 2025)	40.9 (+0.6)	92.6 (-291.4)	75.9%	45.4	3.29x	25.6 (-19.5)	103.5 (-280.5)	73.1%	34.7	2.39x
	WINO (Hong et al., 2026)	37.8 (-2.4)	54.2 (-329.8)	85.9%	77.0	5.60x	-	-	-	-	-
C ⁴	42.1 (+1.8)	39.4 (-344.6)	89.7%	120.0	8.69x	43.3 (-1.8)	29.2 (-354.8)	92.4%	146.0	10.04x	
MBPP (n=257, sequence length=384) (block=32, 12 blocks)	Baseline	45.9	384.0	0.0%	15.4	1.00x	56.0	384.0	0.0%	15.6	1.00x
	Prophet (Li et al., 2026)	1.9 (-44.0)	32.1 (-351.9)	91.6%	73.2	4.78x	0.8 (-55.3)	1.4 (-382.6)	90.6%	433.7	27.82x
	SchED (Mohamed et al., 2026b)	27.6 (-18.3)	66.6 (-317.4)	82.6%	101.3	6.60x	1.9 (-54.1)	16.3 (-367.7)	95.8%	51.1	26.54x
	SlowFast (Wei et al., 2026)	41.2 (-4.7)	21.6 (-362.4)	94.4%	362.0	23.62x	46.7 (-9.3)	30.0 (-354.0)	92.2%	274.1	17.58x
	KLASS (Kim et al., 2025)	46.5 (+0.6)	69.2 (-314.8)	82.0%	66.8	4.35x	53.0 (-3.0)	93.1 (-290.9)	75.8%	42.3	2.71x
	WINO (Hong et al., 2026)	47.1 (+1.2)	54.4 (-329.6)	85.8%	88.0	5.79x	-	-	-	-	-
C ⁴	45.9 (+0.0)	33.3 (-350.7)	91.3%	170.8	11.11x	56.0 (+0.0)	25.5 (-358.5)	93.4%	209.3	13.42x	

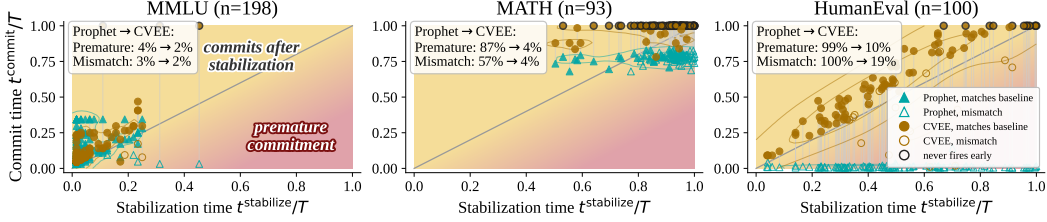


Figure 4: Exit-stabilization phase diagram. Each faint line pairs Prophet and CVEE on the same LLaDA-8B-Instruct trajectory. On MMLU, both often trigger near answer stabilization; on MATH and HumanEval, Prophet frequently triggers before the final answer has stabilized, whereas CVEE waits substantially longer. The comparison tests how closely each stopping signal follows the answer trajectory itself.

τ_{CCTC} before degrading under more aggressive commitment. Increasing H from 0 to 1 improves TPS by 13.1%, 5.4%, and 2.4% on HumanEval, GSM8K, and MMLU, respectively, without a monotonic accuracy loss. Appendix F reports the full sweeps and shows that the fixed setting $(\tau_{\text{CCTC}}, H) = (0.8, 1)$ lies in a broad, conservative region rather than at a task-specific optimum.

Effect of CCTC commitments on CVEE’s exit decision. We repeat decoding while preventing CCTC from making additional commitments inside the candidate span observed by CVEE. On MMLU and GSM8K, across both backbones, the graded answers remain unchanged while the exit step changes only modestly. A stricter control that protects the entire candidate block likewise changes exit timing without a systematic change in the accepted answer ($p = 0.69$ pooled). These results indicate that, for short-span answers, CCTC can accelerate the region around the candidate without determining the answer ultimately accepted by CVEE. For whole-program answers, the two mechanisms are less separable, as analyzed in Appendix C.

5.2 EFFECT OF COMBINING CVEE WITH AN INDEPENDENT SAMPLER

A natural alternative is to combine CVEE with an independently designed adaptive sampler. We test this with WINO (Hong et al., 2026), leaving its local commitment rule unchanged and adding only CVEE’s global stopping decision. On MMLU and HellaSwag, CVEE removes a further 70–76% of WINO’s remaining steps with essentially unchanged accuracy. The same composition is less effective on long-reasoning and code tasks, where WINO’s commitments affect the trajectory from which CVEE evaluates answer stability.

Table 2: CVEE combined with WINO. CVEE provides substantial additional step reduction on short-answer tasks, whereas on long-reasoning and code tasks, preserving accuracy allows only limited additional step reduction.

Task	WINO		+ CVEE		Conservative CVEE	
	Acc	Step	Acc	Step	Acc	Step
MMLU	61.0	16.8	61.5	4.1	–	–
HellaSwag	73.0	12.9	73.0	3.9	–	–
GSM8K	72.0	49.2	36.0	43.6	70.0	48.5
SVAMP	85.0	45.6	62.0	41.7	83.0	45.1
HumanEval	37.8	54.2	14.6	29.7	36.6	50.1

Table 2 shows that recovering accuracy in this regime largely removes the additional benefit of early termination. On GSM8K, for example, a more conservative CVEE setting recovers most of WINO’s accuracy while reducing the mean step count only from 49.2 to 48.5. Similar behavior appears on SVAMP and HumanEval.

These results motivate jointly designing local commitment and global termination. An independently designed commitment rule can change the evidence later used for the stopping decision, whereas C^4 coordinates the two mechanisms explicitly. Appendix E provides the complete results.

5.3 EFFICIENCY DECOMPOSITION AND COMPONENT ABLATION

Effect of CVEE and CCTC components. Table 3 isolates the contribution of each component. Confidence alone is unreliable on HumanEval, reducing accuracy from 40.2 to 4.9 on LLaDA, while stability alone can fail on long-reasoning tasks, reducing GSM8K from 66.0 to 33.0. Combining confidence and stability recovers full-decoding accuracy on GSM8K but removes few steps, indicating that CVEE waits when the answer remains unsettled. CCTC instead reduces computation within blocks that still need to be decoded, while one-step confirmation improves robustness on the harder code setting with little additional cost. The full system combines these complementary effects. Appendix C further examines how CCTC commitments affect CVEE’s exit decision.

Task	Variant	LLaDA-8B-Instruct					Dream-7B-Instruct				
		Acc (%)	Step	Saving	TPS	Speedup	Acc (%)	Step	Saving	TPS	Speedup
General / short-answer tasks											
MMLU (n=200, sequence length=64) (block=16, 4 blocks)	Baseline	64.0	64.0	0.0%	30.4	1.00×	71.5	64.0	0.0%	28.2	1.00×
	+ confidence gate only	64.0	3.7	94.1%	636.8	20.88×	70.5	2.9	95.5%	277.2	9.87×
	+ stability gate only	62.0	4.0	93.7%	525.7	17.18×	70.0	3.7	94.2%	417.0	14.84×
	+ CVEE (joint gate)	64.0	7.0	89.1%	452.7	14.80×	69.5	8.0	87.5%	208.9	7.44×
	+ CCTC, core only	63.5	5.2	91.9%	496.1	16.27×	70.5	5.2	91.9%	323.3	11.54×
	+ CCTC, core then confirm	64.0	5.0	92.1%	491.1	16.10×	70.0	4.7	92.6%	386.4	13.75×
	C ⁴ : CCTC on every block	64.0	4.8	92.5%	490.4	16.16×	70.0	4.5	93.0%	389.7	13.82×
Long-reasoning tasks (multi-step CoT)											
GSM8K (n=100, sequence length=256) (block=32, 8 blocks)	Baseline	66.0	256.0	0.0%	19.0	1.00×	85.0	256.0	0.0%	21.2	1.00×
	+ confidence gate only	61.0	59.2	76.9%	20.7	1.09×	83.0	53.8	79.0%	21.3	1.01×
	+ stability gate only	33.0	24.5	90.4%	50.0	2.63×	83.0	53.2	79.2%	23.3	1.10×
	+ CVEE (joint gate)	66.0	250.0	2.3%	19.2	1.01×	85.0	256.0	0.0%	21.1	1.00×
	+ CCTC, core only	66.0	85.2	66.7%	51.8	2.72×	85.0	85.8	66.5%	63.7	3.01×
	+ CCTC, core then confirm	66.0	83.9	67.2%	52.0	2.73×	85.0	84.3	67.1%	64.6	3.05×
	C ⁴ : CCTC on every block	64.0	62.5	75.6%	67.5	3.55×	85.0	54.5	78.7%	102.1	4.82×
Code generation (whole-program answer)											
HumanEval (n=164, sequence length=384) (block=32, 12 blocks)	Baseline	40.2	384.0	0.0%	13.8	1.00×	45.1	384.0	0.0%	14.5	1.00×
	+ confidence gate only	43.9	9.7	97.5%	67.6	4.89×	5.5	9.5	97.5%	196.5	13.53×
	+ stability gate only	42.1	39.4	89.7%	24.8	1.79×	43.3	29.2	92.4%	43.9	3.02×
	+ CVEE (joint gate)	37.2	212.4	44.7%	24.8	1.79×	42.7	109.2	71.6%	39.3	2.70×
	+ CCTC, core only	40.9	50.0	87.0%	90.8	6.57×	43.3	36.6	90.5%	116.5	8.02×
	+ CCTC, core then confirm	42.1	48.8	87.3%	92.6	6.70×	43.3	35.8	90.7%	119.0	8.19×
	C ⁴ : CCTC on every block	42.1	39.4	89.7%	120.0	8.69×	43.3	29.2	92.4%	146.0	10.04×

Table 3: Component ablation. Confidence- and stability-only rows isolate the two CVEE signals, while the CCTC rows compare boundary-anchored commitment with and without one-step confirmation. The highlighted row is the full C⁴ configuration with CCTC enabled on every block.

Effect on per-block step usage. CCTC reduces computation across multiple blocks rather than concentrating the remaining cost in a single block. Figures 10 and 11 report the per-block step usage across all 12 tasks and both backbones.

L	B	N	Baseline	C ⁴					+ DualCache					Fast-dLLM (DualCache+parallel)				
				Acc	Step	Saving	TPS	Speedup	Acc	Step	Saving	TPS	Speedup	Acc	Step	Saving	TPS	Speedup
HumanEval (code generation), block length swept at fixed generation length 384																		
384	16	24	41.5	39.6	44.5	88.4%	104.5	7.56×	32.3	45.4	88.2%	127.1	9.22×	39.0	64.4	83.2%	102.2	7.41×
384	32	12	40.2	42.1	39.4	89.7%	120.0	8.69×	34.8	44.7	88.4%	154.3	11.20×	37.8	57.2	85.1%	132.2	9.58×
384	64	6	39.6	42.1	36.5	90.5%	128.7	9.30×	36.6	42.2	89.0%	137.9	10.01×	36.6	51.1	86.7%	143.1	10.38×
GSM8K (long-reasoning), generation length swept at fixed block length 32																		
128	32	4	66.0	67.0	44.7	65.1%	77.3	2.51×	63.0	48.6	62.1%	51.7	1.70×	68.0	58.4	54.4%	45.0	1.48×
256	32	8	66.0	64.0	62.5	75.6%	67.5	3.55×	66.0	68.0	73.5%	75.1	3.97×	66.0	80.4	68.6%	63.6	3.36×
512	32	16	72.0	72.0	81.8	84.0%	75.8	5.84×	75.0	88.0	82.8%	108.2	8.34×	73.0	101.9	80.1%	92.4	7.13×
GSM8K (long-reasoning), L and B scaled together at fixed block count 8, so r is held constant																		
128	16	8	65.0	68.0	47.2	63.1%	73.2	2.39×	66.0	52.4	59.0%	54.5	1.79×	66.0	60.8	52.5%	44.8	1.47×
256	32	8	66.0	64.0	62.5	75.6%	67.5	3.55×	66.0	68.0	73.5%	75.1	3.97×	66.0	80.4	68.6%	63.6	3.36×
512	64	8	70.0	77.0	76.8	85.0%	77.2	5.94×	72.0	80.3	84.3%	115.4	8.90×	74.0	97.0	81.1%	99.8	7.69×
MMLU (short-answer), generation length swept at fixed block length 16																		
16	16	1	56.5	58.0	4.0	75.3%	128.3	3.83×	55.5	4.5	72.1%	82.9	2.80×	55.0	6.2	61.4%	59.3	1.96×
32	16	2	61.5	62.5	4.0	87.5%	247.2	7.37×	59.0	4.6	85.6%	147.9	4.94×	59.5	7.5	76.7%	93.3	3.12×
64	16	4	64.0	64.0	4.8	92.5%	490.4	16.16×	60.0	5.1	92.1%	355.5	12.07×	59.5	20.2	68.4%	77.8	2.66×

Table 4: Scaling across generation and block shapes. L is generation length, B is block length, and $N=L/B$ is the number of blocks; Baseline denotes full decoding. C⁴ savings grow with generation length. DualCache provides complementary per-pass acceleration; approximate bidirectional caching may change accuracy and step count relative to uncached C⁴. The backbone is LLaDA-8B-Instruct.

Scaling along each dimension. Table 4 shows that savings grow most clearly with generation length: once the answer stabilizes, additional decoding budget becomes removable slack. Changing block shape has a smaller effect because it changes how local work is partitioned rather than when the answer settles. On MMLU, increasing generation length mostly adds blocks that CVEE never reaches. DualCache is complementary—it reduces the cost of forward passes, whereas C⁴ reduces how many passes execute—but the composition is not exact. On a bidirectional backbone, cached states approximate the full forward pass and can perturb predictions and commit timing; its throughput benefit is clearest on longer sequences.

6 CONCLUSION

C⁴ coordinates two distinct decisions in diffusion decoding: *where* predictions should be committed locally and *when* generation should terminate globally. CVEE tracks the re-extracted answer trajectory and stops only after confidence and temporal stability agree, while CCTC preserves parallel prediction but imposes an autoregressive-style order on additional commitments through a boundary-anchored core and one-step confirmation. Across 12 zero-shot tasks and two backbones, one fixed numerical configuration removes 64–95% of decoding steps and achieves 2.6×–18.6× measured end-to-end speedup while maintaining accuracy comparable to full decoding. These results show that local commitment and global termination are most effective when designed jointly rather than treated as independent acceleration mechanisms.

REFERENCES

- Jacob Austin, Daniel D Johnson, Jonathan Ho, Daniel Tarlow, and Rianne Van Den Berg. Structured denoising diffusion models in discrete state-spaces. *Advances in Neural Information Processing Systems (NeurIPS)*, 34:17981–17993, 2021a.
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021b.
- Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2020.
- Andrew Campbell, Joe Benton, Valentin De Bortoli, Thomas Rainforth, George Deligiannidis, and Arnaud Doucet. A continuous time framework for discrete denoising models. *Advances in Neural Information Processing Systems (NeurIPS)*, 35:28266–28279, 2022.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Yuzhe Chen, Jiale Cao, Xuyang Liu, Jin Xie, Aiping Yang, and Yanwei Pang. STDec: Spatio-temporal stability guided decoding for dLLMs. *arXiv preprint arXiv:2604.06330*, 2026.
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. URL <https://arxiv.org/abs/2110.14168>.
- Muzhi Dai, Chenxu Yang, and Qingyi Si. S-GRPO: Early exit via reinforcement learning in reasoning models. In *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025. URL <https://openreview.net/forum?id=wNMK5o0Vfg>.
- Zhenbang Du, Kejing Xia, Xinrui Zhong, Yonggan Fu, Nicolai Oswald, Binfei Ji, Brucek Khailany, Pavlo Molchanov, and Yingyan Lin. R^2 -dLLM: Accelerating diffusion large language models via spatio-temporal redundancy reduction. *arXiv preprint arXiv:2604.18995*, 2026.
- Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided language models. *International Conference on Machine Learning (ICML)*, 2023.
- Jiawei Gu, Ziyue Qiao, and Xiao Luo. Detecting the semantic fixed point: A geometric framework for efficient inference. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. URL <https://openreview.net/forum?id=DACN5xM4h7>.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations (ICLR)*, 2021a.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS Datasets and Benchmarks Track (NeurIPS)*, 2021b.
- Feng Hong et al. Wide-in, narrow-out: Revokable decoding for efficient and effective DLLMs. In *International Conference on Learning Representations (ICLR)*, 2026. URL <https://openreview.net/pdf?id=XtLQH1NLxy>.
- Emiel Hoogetboom, Didrik Nielsen, Priyank Jaini, Patrick Forré, and Max Welling. Argmax flows and multinomial diffusion: Learning categorical distributions. *Advances in Neural Information Processing Systems (NeurIPS)*, 34:12454–12465, 2021.

- Jiyeon Kim, Sungik Choi, Yongrae Jo, Moontae Lee, and Minjoon Seo. Early decisions matter: Proximity bias and initial trajectory shaping in non-autoregressive diffusion language models. *arXiv preprint arXiv:2604.10567*, 2026. URL <https://arxiv.org/abs/2604.10567>.
- Seo Hyun Kim, Sunwoo Hong, Hojung Jung, Youngrok Park, and Se-Young Yun. KLASS: KL-guided fast inference in masked diffusion models. In *Advances in Neural Information Processing Systems 38 (NeurIPS), Spotlight*, 2025. arXiv:2511.05664.
- Jungseob Lee, Seongtae Hong, Seungjun Lee, Jaehyung Seo, Junyoung Son, Sugyeong Eo, Chanjun Park, Hyeongju Park, Hyeonseok Moon, and Heuiseok Lim. DART: Draft-agreement routing for training-free adaptive thinking budgets in hybrid reasoning models. *arXiv preprint arXiv:2606.23181*, 2026. URL <https://arxiv.org/abs/2606.23181>.
- Pengxiang Li, Yefan Zhou, Dilxat Muhtar, Lu Yin, Shilin Yan, Li Shen, Yi Liang, Soroush Vosoughi, and Shiwei Liu. Diffusion language model knows the answer before it decodes. In *The Fourteenth International Conference on Learning Representations (ICLR)*, 2026. URL <https://openreview.net/forum?id=g88nt4ieTG>.
- Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pp. 32819–32848, 2024. URL <https://proceedings.mlr.press/v235/lou24a.html>.
- Shen-Yun Miao, Chao-Chun Liang, and Keh-Yih Su. A diverse corpus for evaluating and developing English math word problem solvers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.
- Amr Mohamed, Guokan Shang, and Michalis Vazirgiannis. LESS is more: Mutual-stability sampling for diffusion language models. *arXiv preprint arXiv:2606.16908*, 2026a. URL <https://arxiv.org/abs/2606.16908>.
- Amr Mohamed, Yang Zhang, Michalis Vazirgiannis, and Guokan Shang. Fast-decoding diffusion language models via progress-aware confidence schedules. In *Findings of the Association for Computational Linguistics: ACL 2026*, 2026b. arXiv:2512.02892.
- Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. In *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025. URL <https://openreview.net/forum?id=KnqiC0znVF>. arXiv:2502.09992.
- Jingyang Ou, Shen Nie, Kaiwen Xue, Fengqi Zhu, Jiacheng Sun, Zhenguo Li, and Chongxuan Li. Your absorbing discrete diffusion secretly models the conditional distributions of clean data. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. URL <https://openreview.net/forum?id=sMyXP8Tanm>.
- Arkil Patel, Satwik Bhattamishra, and Navin Goyal. Are NLP models really able to solve simple math word problems? In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2021.
- Jiaoyang Ruan, Xin Gao, Yinda Chen, Hengyu Zeng, Liang Du, Guanghao Li, Jie Fu, and Jian Pu. Reasoning on the manifold: Bidirectional consistency for self-verification in diffusion language models. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. URL <https://openreview.net/forum?id=CUVBdw2tKy>. arXiv:2604.16565.
- Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T. Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/eb0b13cc515724ab8015bc978fdde0ad-Abstract-Conference.html.
- Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. WinoGrande: An adversarial Winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106, 2021.

- Jiaxin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, and Michalis K. Titsias. Simplified and generalized masked diffusion for discrete data. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/bad233b9849f019aead5e5cc60cef70f-Abstract-Conference.html.
- Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *International Conference on Machine Learning (ICML)*, pp. 2256–2265, 2015.
- Chengcheng Wang, Tingzhang Luo, Wenhao Li, Jianyuan Guo, and Chang Xu. TACG: Trajectory-aware commit gating for diffusion language model decoding. *arXiv preprint arXiv:2607.03236*, 2026.
- Qingyan Wei, Yaojie Zhang, Zhiyuan Liu, Puyu Zeng, Yuxuan Wang, Biqing Qi, Dongrui Liu, and Linfeng Zhang. Accelerating diffusion large language models with SlowFast sampling: The three golden principles. In *The Fourteenth International Conference on Learning Representations (ICLR)*, 2026. URL <https://openreview.net/forum?id=Uh17FiwF4q>. arXiv:2506.10848.
- Chengyue Wu, Hao Zhang, Shuchen Xue, Zhijian Liu, Shizhe Diao, Ligeng Zhu, Ping Luo, Song Han, and Enze Xie. Fast-dLLM: Training-free acceleration of diffusion LLM by enabling KV cache and parallel decoding. In *The Fourteenth International Conference on Learning Representations (ICLR)*, 2026. URL <https://openreview.net/forum?id=3Z3Is6hn0T>. arXiv:2505.22618.
- Andrea Wynn, Metod Jazbec, Charith Peris, Rinat Khaziev, Anqi Liu, Daniel Khashabi, and Eric Nalisnick. Controlling the risk of corrupted contexts for language models via early-exiting. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. URL <https://openreview.net/forum?id=8bUDbWMo5v>. arXiv:2510.02480.
- Yangxinyu Xie, Tao Wang, Soham Mallick, Yan Sun, Georgy Noarov, Mengxin Yu, Tanwi Mallick, Weijie J. Su, and Edgar Dobriban. Statistical early stopping for reasoning models. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. URL <https://openreview.net/forum?id=TJshRZDdyW>. arXiv:2602.13935.
- Chenxu Yang, Qingyi Si, Yongjie Duan, Zheliang Zhu, Chenyu Zhu, Qiaowei Li, Minghui Chen, Zheng Lin, and Weiping Wang. Dynamic early exit in reasoning models. In *The Fourteenth International Conference on Learning Representations (ICLR)*, 2026. URL <https://openreview.net/forum?id=NpU7ZXafRi>.
- Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7B: Diffusion large language models. *arXiv preprint arXiv:2508.15487*, 2025. URL <https://arxiv.org/abs/2508.15487>.
- Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019.
- Kevin Zhai, Sabbir Mollah, Zhenyi Wang, and Mubarak Shah. CORE: Context-robust remasking for diffusion language models. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. URL <https://openreview.net/forum?id=bmKHxLWkz9>. arXiv:2602.04096.
- Yu Zhang, Xinchun Li, Jiale Zhou, Hongnan Ma, Zhongwei Wan, Yiwei Shi, Duoqian Miao, Qi Zhang, and Longbing Cao. Swordsman: Entropy-driven adaptive block partition for efficient diffusion language models. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. arXiv:2602.04399.
- Kaiwen Zheng, Yongxin Chen, Hanzi Mao, Ming-Yu Liu, Jun Zhu, and Qinsheng Zhang. Masked diffusion models are secretly time-agnostic masked models and exploit inaccurate categorical sampling. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. URL <https://openreview.net/forum?id=CTC7CmirNr>.

APPENDIX OVERVIEW

The appendices provide additional details on reproducibility, mechanism analysis, robustness, and output quality. Appendices A and B specify the experimental configuration and end-to-end timing protocol. Appendices C and D analyze the interaction between local commitment and global termination and illustrate the behavior of different exit criteria on real decoding trajectories. Appendices E and F study composition with an independent sampler and sensitivity to CCTC design choices. Appendix G examines the quality of text produced after early exit, and Appendix H summarizes the remaining limitations.

- **A** Experimental configuration and hyperparameters.
- **B** End-to-end timing protocol.
- **C** Effect of CCTC commitments on C_{VEE}’s exit decision.
- **D** Why Prophet exits too early on real trajectories.
- **E** Effect of composing C_{VEE} with an independent sampler.
- **F** Sensitivity analysis.
- **G** Text quality after early exit.
- **H** Limitations.

A EXPERIMENTAL CONFIGURATION AND HYPERPARAMETERS

Evaluation setup. All experiments use zero-shot prompting. Reasoning tasks additionally receive the prompt “Let’s think step by step,” with no demonstrations or additional answer-format instructions. Dream-7B-Instruct uses its official one-position logit shift for next-token indexing; all other decoding settings follow the same evaluation protocol as LLaDA-8B-Instruct. For Prophet, we implement its published stopping rule in the same evaluation pipeline rather than importing reported numbers obtained under a different prompt template.

Fixed C⁴ configuration. The same five numerical hyperparameters are used for every task and both backbones: C_{VEE} uses $\tau_{\text{C_{VEE}}}=0.7$, $\gamma=2.0$, and $p_{\text{min}}=3$, while CCTC uses $\tau_{\text{CCTC}}=0.8$ and gap budget $H=1$. The extractor additionally uses `tail_frac=0.3` when `search_mode = last`. No numerical parameter of C⁴ is adjusted by task or backbone. The extractor’s search region changes only with the output format, as described below.

Task	n	Len.	Steps	Block	search_mode	Prophet ($\tau_{\text{high}}/\tau_{\text{mid}}/\tau_{\text{low}}$)	SchED (mode/patience)	KLASS ($\tau/\epsilon_{\text{KL}}$)	WINO (τ/τ_{back})
MMLU	200	64	64	16	first5	7.5/5.0/2.5	cosine / 0	0.7/0.015	0.5/0.9
ARC-C	200	64	64	16	first5	7.5/5.0/2.5	cosine / 0	0.7/0.015	0.5/0.9
HellaSwag	200	64	64	16	first5	7.5/5.0/2.5	cosine / 0	0.7/0.015	0.5/0.9
WinoGrande	200	64	64	16	first5	7.5/5.0/2.5	cosine / 0	0.7/0.015	0.5/0.9
PIQA	200	64	64	16	first5	7.5/5.0/2.5	cosine / 0	0.7/0.015	0.5/0.9
GSM8K	100	256	256	32	last	8.0/5.0/3.5	linear / 4	0.6/0.015	0.6/0.9
MATH	100	256	256	32	last	8.0/5.0/3.5	linear / 4	0.6/0.015	0.7/0.9
SVAMP	100	256	256	32	last	8.0/5.0/3.5	linear / 4	0.6/0.015	0.6/0.9
ASDiv	100	256	256	32	last	8.0/5.0/3.5	linear / 4	0.6/0.015	0.6/0.9
GSM-Hard	100	256	256	32	last	8.0/5.0/3.5	linear / 4	0.6/0.015	0.6/0.9
HumanEval	164	384	384	32	code	7.5/5.0/4.5	cosine / 4	0.9/0.01	0.7/0.9
MBPP	257	384	384	32	code	7.5/5.0/4.5	cosine / 4	0.7/0.01	0.8/0.9

Table 5: Task settings used in Table 1. n is the number of evaluated examples, Len. is the generation length, Steps is the full decoding budget, and Block is the block length. `search_mode` is determined by the expected output format. Peer settings follow their published or released configurations; model-specific exceptions are described below.

Peer configurations. Prophet uses its published staged confidence thresholds, including the separate schedule reported for code generation. SchED uses its calibrated decay schedule and patience value together with Prophet’s corresponding high and low thresholds. For code generation, where SchED does not provide a dedicated configuration, we use its long-form stability guard because it gives the stronger result on both backbones.

KLASS uses its calibrated task-family settings, with the released code-generation configurations used for HumanEval and MBPP. These code settings are model-specific: on HumanEval they are 0.9/0.01 for LLaDA and 0.8/0.001 for Dream, and on MBPP they are 0.7/0.01 and 0.9/0.001, respectively.

WINO uses its published commitment threshold where available and the nearest published task-family setting otherwise; $\tau_{\text{back}}=0.9$ throughout. Fast-dLLM uses its released implementation with threshold 0.9 and dual KV caching enabled.

Extractor search regions. CVEE assumes only that the task output contains an identifiable answer unit that can be deterministically extracted; its position need not be fixed across decoding steps. This is a weaker location assumption than prior DLM early-exit methods such as Prophet (Li et al., 2026) and SchED (Mohamed et al., 2026b), which monitor confidence over a predefined answer region or response span. CVEE instead re-extracts and relocates the candidate at every step within a format-defined search region.

For GSM8K, MATH, SVAMP, ASDiv, and GSM-Hard, `search_mode = last` restricts extraction to the final `tail_frac=0.3` of the generation. For MMLU, ARC-C, HellaSwag, WinoGrande, PIQA, and TruthfulQA, `search_mode = first5` searches for the answer within the first five output positions. For HumanEval and MBPP, `search_mode = code` tracks the generated function body, whose end is identified by Python indentation. Character-level spans are mapped back to token indices so that multi-line code does not depend on detokenize–retokenize boundary preservation.

B TIMING PROTOCOL

Step reduction does not directly determine wall-clock speedup because different methods incur different computation within each decoding step. CVEE performs candidate extraction and gate evaluation, while CCTC changes the set of positions committed during decoding. We therefore measure throughput directly rather than estimating it from the number of executed steps.

All RTX 4090 TPS and Speedup results in the main experiments and ablations are measured end-to-end over the decoding loop. We call `torch.cuda.synchronize()` immediately before and after each timed decode and measure elapsed host time with `time.time()`. The same procedure is used for Baseline, Prophet Li et al. (2026), SlowFast Sampling (Wei et al., 2026), SchED (Mohamed et al., 2026b), KCLASS (Kim et al., 2025), WINO (Hong et al., 2026) and C^4 .

Measurement procedure. Each method is timed on the same fixed set of 20 evaluation examples. Two additional examples are decoded as warmup and excluded from measurement. We then perform three complete timing runs over the 20 examples and report the mean throughput across the three runs. Experiments use batch size 1, bf16, greedy decoding, and an otherwise idle NVIDIA GeForce RTX 4090.

Setting	Value
GPU	NVIDIA GeForce RTX 4090
Batch size	1
Precision	bf16
PyTorch / CUDA	2.5.1+cu121 / 12.1
Synchronization	<code>torch.cuda.synchronize()</code> before and after each timed decode
Timing	Host wall-clock time with <code>time.time()</code>
Examples / warmup / repeats	20 / 2 / 3
LLaDA timing CV	0.00–2.98% (mean 0.14%)
Dream timing CV	0.00–0.48% (mean 0.08%)

Table 6: RTX 4090 timing protocol. CV denotes the coefficient of variation across the three repeated timing runs for each measured configuration.

The low variation across repeated runs indicates that the reported throughput differences are not dominated by timing noise. Step and TPS should nevertheless be interpreted separately: two methods may execute the same number of decoding steps while differing in wall-clock throughput because the amount of computation performed within those steps can differ.

C EFFECT OF CCTC COMMITMENTS ON CVEE’S EXIT DECISION

CCTC accelerates the same blocks from which CVEE extracts its candidate, raising a potential coupling: commitments made by CCTC inside or around the candidate span could make the answer appear stable and thereby influence CVEE’s exit decision. We test this directly by selectively removing

CCTC’s additional commits while leaving the reference decoder’s scheduled top- k commitments unchanged.

Effect of Protecting the Candidate Span. We first prevent CCTC from making additional commits inside the candidate span observed by CVEE. Table 7 reports four diagnostics. *Frozen* is the fraction of the accepted span already committed when CVEE fires, and *By CCTC* is the fraction committed specifically by CCTC rather than by the reference decoder’s scheduled top- k rule. *Recon. tok.* and *Recon. exact* re-mask the accepted span on a copy of the current buffer and measure whether one additional forward pass reconstructs the same tokens or the entire span, respectively. *Same ans.* is the fraction of examples whose graded answer remains unchanged when CCTC is barred from committing inside the candidate span.

Task	LLaDA-8B-Instruct					Dream-7B-Instruct				
	Frozen	By CCTC	Recon. tok.	Recon. exact	Same ans.	Frozen	By CCTC	Recon. tok.	Recon. exact	Same ans.
MMLU	68.1%	37.7%	97.9%	97.9%	100.0%	74.6%	50.3%	97.9%	97.9%	100.0%
GSM8K	11.5%	7.7%	100.0%	100.0%	100.0%	0.0%	0.0%	100.0%	100.0%	100.0%
HumanEval	99.9%	86.3%	33.8%	6.8%	59.1%	99.7%	75.1%	32.2%	12.5%	72.0%

Table 7: Effect of protecting the candidate span from CCTC commits. For MMLU and GSM8K, removing CCTC’s additional commits inside the candidate span leaves every graded answer unchanged. HumanEval differs because the candidate is the generated program itself.

For MMLU and GSM8K, protecting the candidate span leaves the graded answer unchanged on every example across both backbones. This remains true even on Dream MMLU, where CCTC accounts for 50.3% of the accepted span before exit. The mean exit step changes by at most 1.2 steps. The reconstruction probe is consistent with the same conclusion: the accepted span is reconstructed exactly on 97.9% of MMLU examples and on all GSM8K examples. Thus, for these short-span answers, CCTC may affect when CVEE reaches its stopping condition without determining the answer that CVEE ultimately accepts.

Effect of Protecting the Entire Candidate Block. Protecting only the candidate span still allows CCTC to commit neighboring positions in the same block, which could indirectly affect the candidate trajectory. We therefore use a stricter control: from the first step at which a candidate is extractable until CVEE fires, CCTC makes no additional commits anywhere in the block containing that candidate. All other blocks retain the deployed rule.

This control delays exit. On MMLU, the mean step count increases from 4.8 to 6.3 for LLaDA and from 4.5 to 7.1 for Dream; on GSM8K, it increases from 62.5 to 82.7 and from 54.5 to 58.6, respectively. The graded answers, however, show no systematic directional change. Answer identity is preserved on 98.5% and 98.0% of MMLU examples and on 95% and 100% of GSM8K examples. Pooled across the four settings, four examples change from wrong to correct and two from correct to wrong ($p = 0.69$, two-sided sign test).

These results indicate that CCTC’s commits inside the candidate block can accelerate when CVEE fires, but provide no evidence that they systematically determine the short-span answer that CVEE accepts.

Effect on Whole-Program Answers. HumanEval is a different regime because the tracked candidate is the generated function itself. At the firing step, nearly the entire candidate has already been committed, and most of those commits come from CCTC. Protecting the candidate span therefore removes a substantial part of CCTC’s effect on the generation. On LLaDA, the protected-span run differs from the deployed run on 41% of examples, accuracy changes from 42.1 to 36.0, and the mean step count rises from 39.4 to 185.4. On Dream, accuracy remains 43.3 while the mean step count rises from 29.2 to 79.4.

The same control is less diagnostic in this regime because protecting a whole-program candidate is close to disabling CCTC over most of the remaining generation. The reconstruction probe is also less informative: re-masking a roughly two-hundred-token program and reconstructing it in one parallel pass succeeds poorly regardless of whether the deployed or protected-span trajectory produced the program. Token reconstruction changes only from 33.8% to 37.7% on LLaDA and from 32.2% to 32.7% on Dream.

We therefore restrict the separability claim to short-span, format-delimited answers. For whole-program answers, CCTC and CVEE remain operationally coupled, and the protected-span control cannot isolate the two decisions without simultaneously removing much of CCTC’s acceleration.

D WHY PROPHET EXITS TOO EARLY ON REAL TRAJECTORIES

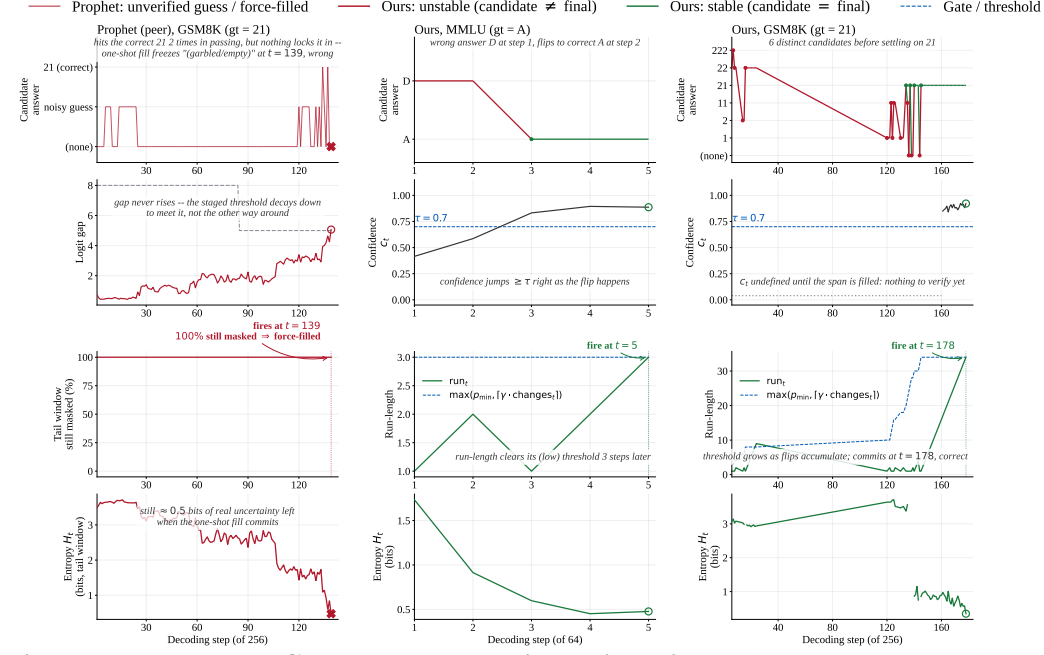


Figure 5: Prophet and CVEE on real decoding trajectories. Columns 1 and 3 use the same GSM8K trajectory, so the two gates are evaluated on identical intermediate predictions rather than on separate runs. Prophet exits at 54% progress while the monitored tail remains masked and produces a non-answer, whereas CVEE waits until 70% and returns the correct answer. Column 2 shows the early-stabilization case on MMLU, where the candidate flips once and CVEE waits for both confidence and stability before firing. Pooled statistics are reported in Figure 2.

Section 4.1 shows in aggregate that progress- or margin-based stopping can fire before the answer has stabilized. This appendix illustrates the same mechanism on individual trajectories. The goal is not to replace the pooled analysis, but to show concretely what each gate is responding to on real decodes.

Effect on a late-stabilizing trajectory. Columns 1 and 3 of Figure 5 replay the same GSM8K decode, so every intermediate prediction is identical and only the stopping rule differs. Prophet exits at step 139 of 256. At that point, its monitored tail window is still masked, and the eventual one-shot fill writes a truncated continuation with no extractable final number. The answer candidate has visited the correct value of 21 in passing, but Prophet does not test whether that value has stabilized.

CVEE on the same trajectory waits until step 178. Once the candidate span becomes available, the candidate oscillates across several values before settling on the correct one. CVEE therefore withholds termination until both conditions of Eq. 4 hold: the span is confident and the extracted answer has remained stable for long enough relative to its earlier flips. This trajectory illustrates the late-stabilization regime of Section 4.1: Prophet reacts to an aggregate signal, while CVEE tracks the answer trajectory itself.

Effect on an early-stabilizing trajectory. Column 2 shows the typical short-answer case on MMLU. The candidate is wrong at step 1, flips to the correct answer at step 2, and its confidence rises sharply at the same time. CVEE still does not fire immediately. It waits until the run-length condition is also satisfied, and then exits at step 5 of 64. This is the intended behavior of the joint gate: confidence alone is insufficient, and a recent flip requires additional stable observations before termination.

Effect on the decoded text. Figure 6 shows the literal decoded buffer for the GSM8K trajectory at each method’s own stopping step. *Baseline* runs to completion and is correct. *Prophet* shares

the same trajectory up to its trigger, but its one-shot fill produces a garbled tail with no extractable number. C⁴ continues decoding longer and commits after the answer has settled, leaving part of the budget unused while preserving the correct final answer.

The same qualitative pattern appears on additional trajectories. Figures 7 and 8 show representative SVAMP and PIQA examples, respectively. Across these cases, the main difference is not threshold strictness alone, but what the gate tests: Prophet uses a region-level aggregate confidence signal, whereas C⁴ verifies the extracted answer itself. Appendix G measures the quality of the one-shot fill more broadly across tasks.

Prompt: Liam and Mitchell own competing lemonade stands across the street from one another. When Liam bragged that he had made \$63 one weekend, Mitchell laughed and told Liam he had sold 21 lemonades at \$4 apiece the same weekend. How many more dollars did Mitchell make selling lemonade that weekend than Liam?

Baseline (256/256 steps)

✓ Correct

To determine how many more dollars Mitchell made compared to Liam, we need to follow these steps:

1. Calculate Mitchell's earnings:

Mitchell sold 21 lemonades. Each lemonade was sold for \$4. Therefore:

$$21 \times 4 = 84 \text{ dollars}$$

2. Determine the difference:

Liam made \$63. Mitchell made \$84.

$$84 - 63 = 21 \text{ dollars}$$

So, Mitchell made \$21 more than Liam.

Prophet (139/256 steps)

✗ Wrong

To determine how many more dollars Mitchell made compared to Liam, we need to follow these steps:

1. Calculate Mitchell's earnings:

Mitchell sold 21 lemonades. Each lemonade was sold for \$4. Therefore:

$$21 \times 4 = 84 \text{ dollars}$$

2. Determine the difference:

Liam made \$63. Mitchell made \$84.

$$84 - 63 = 21 \text{ dollars}$$

So, Mitchell made \$21 more than than..

(one-shot fill froze mid-flicker; no number extractable)

C⁴ (178/256 steps)

✓ Correct

To determine how many more dollars Mitchell made compared to Liam, we need to follow these steps:

1. Calculate Mitchell's earnings:

Mitchell sold 21 lemonades. Each lemonade was sold for \$4. Therefore:

$$21 \times 4 = 84 \text{ dollars}$$

2. Determine the difference:

Liam made \$63. Mitchell made \$84.

$$84 - 63 = 21 \text{ dollars}$$

So, Mitchell made \$21 more dollars than Liam.

(committed here; budget left unspent)

Figure 6: Literal decoded text at each method’s stopping step. Real GSM8K trajectory with ground-truth answer 21, corresponding to Figure 5. The extracted final answer is highlighted in blue. Prophet’s one-shot fill leaves no number to extract.

SVAMP (ground truth = 15)

Baseline (256/256 steps)

✓ Correct

Sure, let's break it down step by step:

1. **Initial number of cakes:**

The baker initially made 155 cakes.

2. **Cakes bought by the friend:**

The friend bought 140 cakes from the baker.

To find out how many cakes the baker still has, we need to subtract the number of cakes bought by the friend from the initial number of cakes.

$$155 - 140 = 15$$

So, the baker still has 15 cakes.

Prophet (92/256 steps)

✗ Wrong

Sure, let's break it down step by step:

1. **Initial number of cakes:**

The baker initially made 155 cakes.

2. **Cakes bought by the friend:**

The friend bought 140 cakes from the baker.

To find out how many cakes the baker still has, we need to subtract the number of cakes bought by the friend from the initial number of cakes.

$$155 - 140 = 15$$

SoSo, the would still still 15 cakes cakes.

(one-shot fill collides two overlapping continuations into one garbled sentence)

C⁴ (256/256 steps)

✓ Correct

Sure, let's break it down step by step:

1. **Initial number of cakes:**

The baker initially made 155 cakes.

2. **Cakes bought by the friend:**

The friend bought 140 cakes from the baker.

To find out how many cakes the baker still has, we need to subtract the number of cakes bought by the friend from the initial number of cakes.

$$155 - 140 = 15$$

So, the baker still has 15 cakes.

(C⁴ never clears its confidence bar early on this example; runs to the full budget and still lands on clean, correct text)

Figure 7: Representative SVAMP trajectory. The same literal-decoded-text comparison as Figure 6. C⁴ correctly declines to stop while the answer trajectory remains unsettled.

PIQA (ground truth = A)

Baseline (64/64 steps)

✓ Correct

A. Pile your dirty clothing items into a zip it compressor bag and place bottom of your suitcase.

Prophet (16/64 steps)

✓ Correct

A. Pile packing dirty clothing items into zip it compressor bag and place in the bottom of your suitcase.

(the extracted letter is still correct, but the one-shot fill drops and reorders words downstream of the answer token itself)

C⁴ (3/64 steps)

✓ Correct

A. Pile your dirty clothing items into a zip it compressor bag and place in the bottom of your suitcase.

(committed after 3 steps; the rest of the buffer is still filled in by the ordinary schedule, not frozen mid-word)

Figure 8: Representative PIQA trajectory. The same literal-decoded-text comparison as Figure 6. Even when Prophet’s extracted answer remains correct, its one-shot fill can still degrade the surrounding text.

E EFFECT OF COMPOSING CVEE WITH AN INDEPENDENT SAMPLER

A natural alternative to C⁴ is to combine CVEE with an independently designed adaptive sampler. We test this with WINO, leaving its local commitment rule unchanged and adding only CVEE’s global stopping decision. WINO controls which positions may commit but does not decide when the entire sequence may terminate, whereas CVEE makes only the latter decision. Because CVEE uses predictions and token probabilities already available at each decoding step, the composition requires no additional forward pass. CCTC is not combined with WINO because both act on local commitment.

Effect on short-answer tasks. On MMLU and HellaSwag, CVEE removes a further 70–76% of WINO’s remaining steps with essentially unchanged accuracy. WINO still leaves several decoding steps after the answer has stabilized in this regime, allowing CVEE to remove the remaining unnecessary computation.

Effect on long-reasoning and code. The same composition is less effective on GSM8K, SVAMP, and HumanEval. With the fixed CVEE configuration, accuracy drops by 23–36 points. We additionally evaluate 45 combinations of $\tau_{\text{CVEE}} \in \{0.7, 0.8, 0.9\}$, $p_{\min} \in \{3, 5, 8, 12, 16\}$, and $\gamma \in \{2, 4, 8\}$ for each task. Recovering accuracy requires substantially more decoding. On GSM8K, for example, a conservative CVEE configuration recovers accuracy from 36.0 to 70.0, but reduces the average step count only from 49.2 to 48.5 relative to WINO.

Task	WINO		+ CVEE		Conservative CVEE	
	Acc	Step	Acc	Step	Acc	Step
<i>Short-answer</i>						
MMLU	61.0	16.8	61.5	4.1	–	–
HellaSwag	73.0	12.9	73.0	3.9	–	–
<i>Long-reasoning and code</i>						
GSM8K	72.0	49.2	36.0	43.6	70.0	48.5
SVAMP	85.0	45.6	62.0	41.7	83.0	45.1
HumanEval	37.8	54.2	14.6	29.7	36.6	50.1

Table 8: Effect of composing CVEE with WINO on LLaDA-8B-Instruct. CVEE provides substantial additional step reduction on short-answer tasks. On long-reasoning and code tasks, more conservative CVEE settings recover most of the lost accuracy but leave only limited additional step reduction.

Effect of WINO commitments on CVEE. WINO has already removed much of the decoding trajectory before CVEE is applied. On GSM8K, WINO reduces the full 256-step budget to 49.2 steps, leaving primarily the portion in which the answer is still changing. By contrast, on MMLU it leaves 16.8 steps while the answer typically stabilizes near the fourth step, leaving substantial opportunity for global termination.

WINO’s local commitments also affect the evidence observed by CVEE. Varying τ_{CVEE} at fixed $(p_{\min}, \gamma)$ does not change accuracy on the three long-output tasks, indicating that confidence is not the limiting condition. Once WINO commits positions inside the answer span, CVEE observes high confidence on predictions that have already been fixed by the sampler, making sampler-induced persistence difficult to distinguish from answer convergence.

This result illustrates the coupling between local commitment and global termination. Appendix C examines the same interaction within C^4 . CCTC instead structures additional commitments through a boundary-anchored core and one-step confirmation while CVEE independently tracks the extracted answer trajectory.

F SENSITIVITY

Effect of the gap budget. The gap budget H controls how many unresolved low-confidence positions CCTC’s boundary-anchored core may cross. At $H=0$, the core is strictly contiguous; increasing H allows the core to cross more unresolved positions, while $H=\infty$ removes structural deferral entirely. We vary $H \in \{0, 1, 2, 4, \infty\}$ on HumanEval, GSM8K, and MMLU while holding all other C^4 settings fixed.

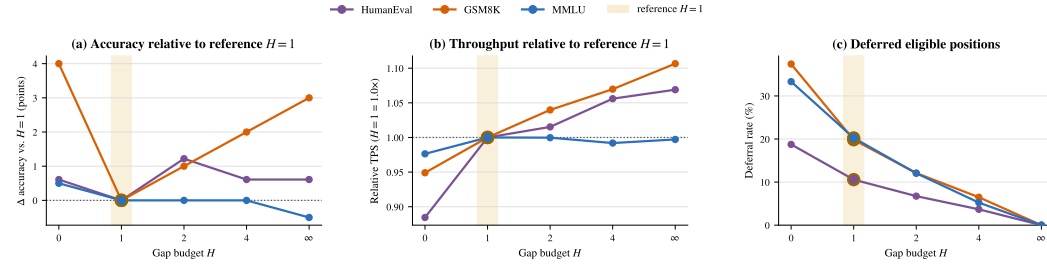


Figure 9: Effect of the gap budget H on LLaDA-8B-Instruct. Accuracy and TPS are normalized to $H=1$. Increasing H reduces structural deferral and generally improves throughput, while accuracy shows no monotonic trend.

Moving from $H=0$ to $H=1$ increases TPS by 13.1%, 5.4%, and 2.4% on HumanEval, GSM8K, and MMLU, respectively, while reducing deferral rates from 18.7/37.4/33.3% to 10.6/19.8/20.2%. Across the full range of H , accuracy differs from the $H=1$ setting by at most 1.2, 4.0, and 0.5 points on the three tasks. We use $H=1$ as a single fixed setting across tasks.

Per-block step usage. Figures 10 and 11 extend the per-block analysis of Section 5.3 to all 12 tasks and both backbones. Short-answer tasks typically terminate within the first one or two blocks, whereas long-reasoning and code tasks retain more computation in later blocks. This explains why CVEE contributes most of the savings on short-answer tasks, while CCTC plays a larger role on longer outputs.

Effect of the CCTC confidence threshold. We vary τ_{CCTC} to examine the sensitivity of CCTC to its confidence threshold. Table 1 uses the same fixed value, $\tau_{\text{CCTC}}=0.8$, for every task and both backbones. We sweep the threshold across all 12 tasks to determine whether this value lies near a narrow accuracy boundary.

These experiments are measured on an NVIDIA A100 rather than the RTX 4090 used for Table 1, so the speedup values should be compared only within each sweep.

Across the short-answer tasks, $\tau_{\text{CCTC}}=0.8$ lies within a broad region of stable accuracy. Long-reasoning and code tasks are more sensitive to aggressive commitment, supporting the use of a single conservative threshold rather than task-specific tuning.

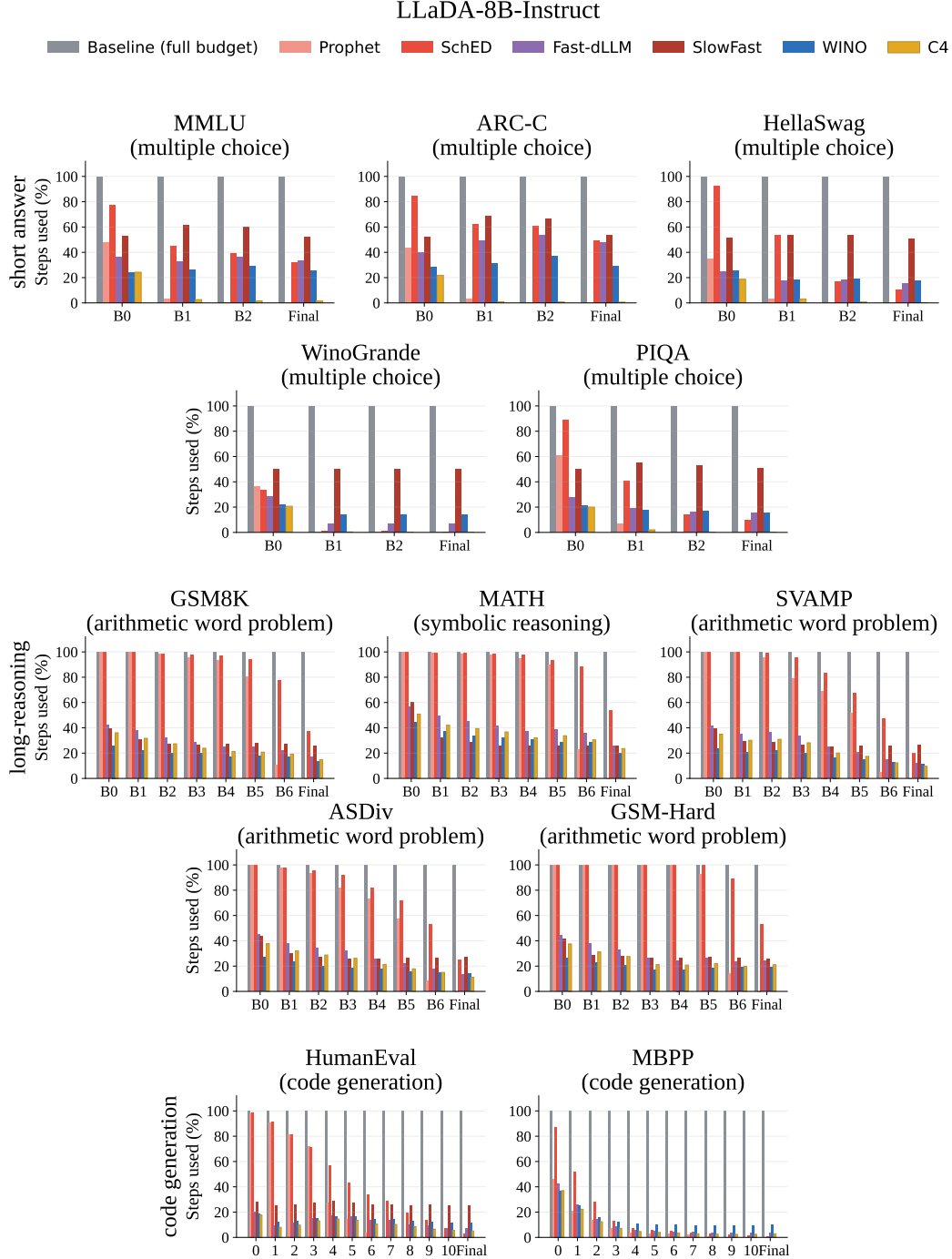
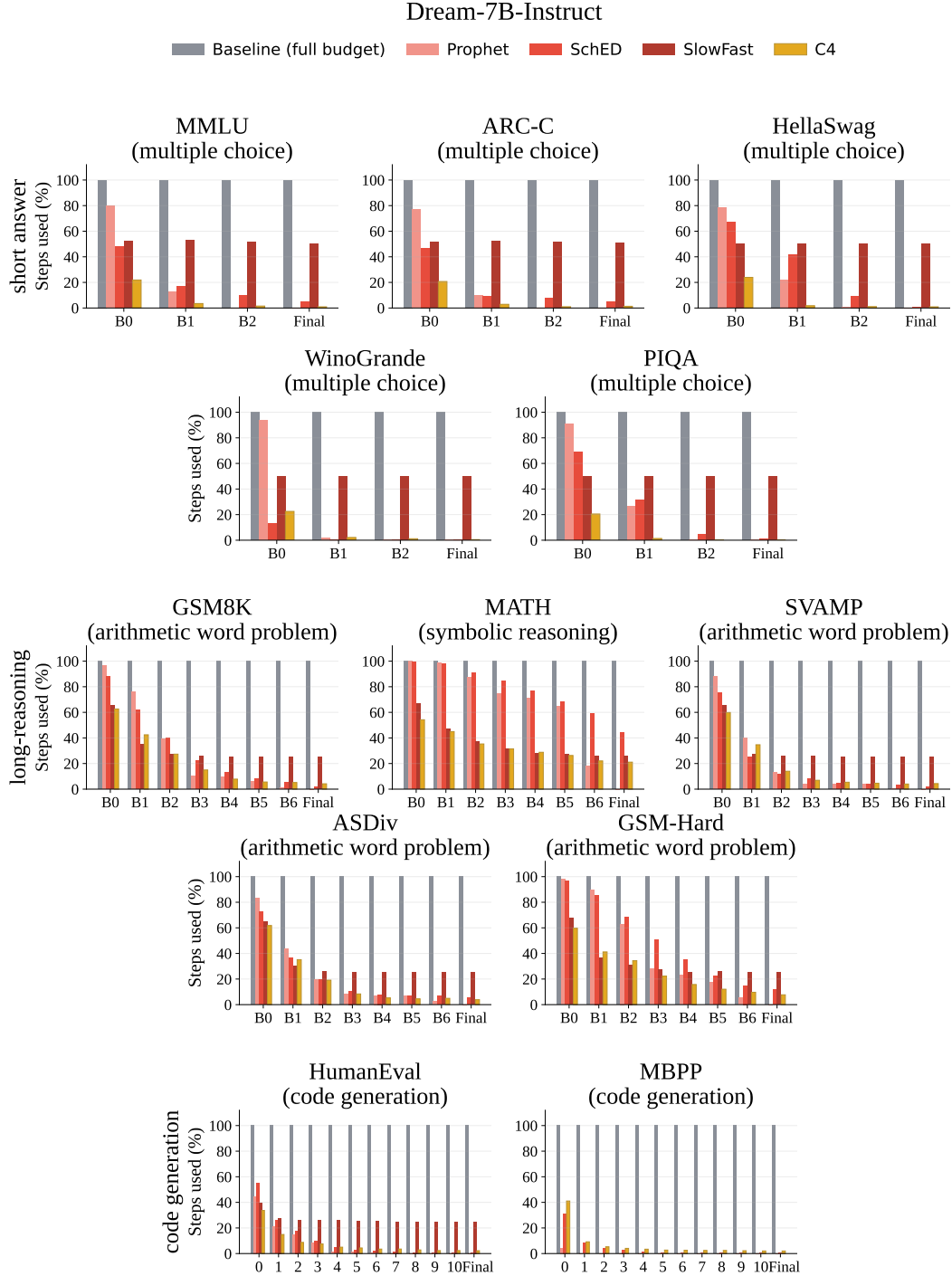


Figure 10: Per-block step usage across all 12 tasks, LLaDA-8B-Instruct. Short-answer tasks terminate early, whereas long-reasoning and code tasks retain more within-block computation.



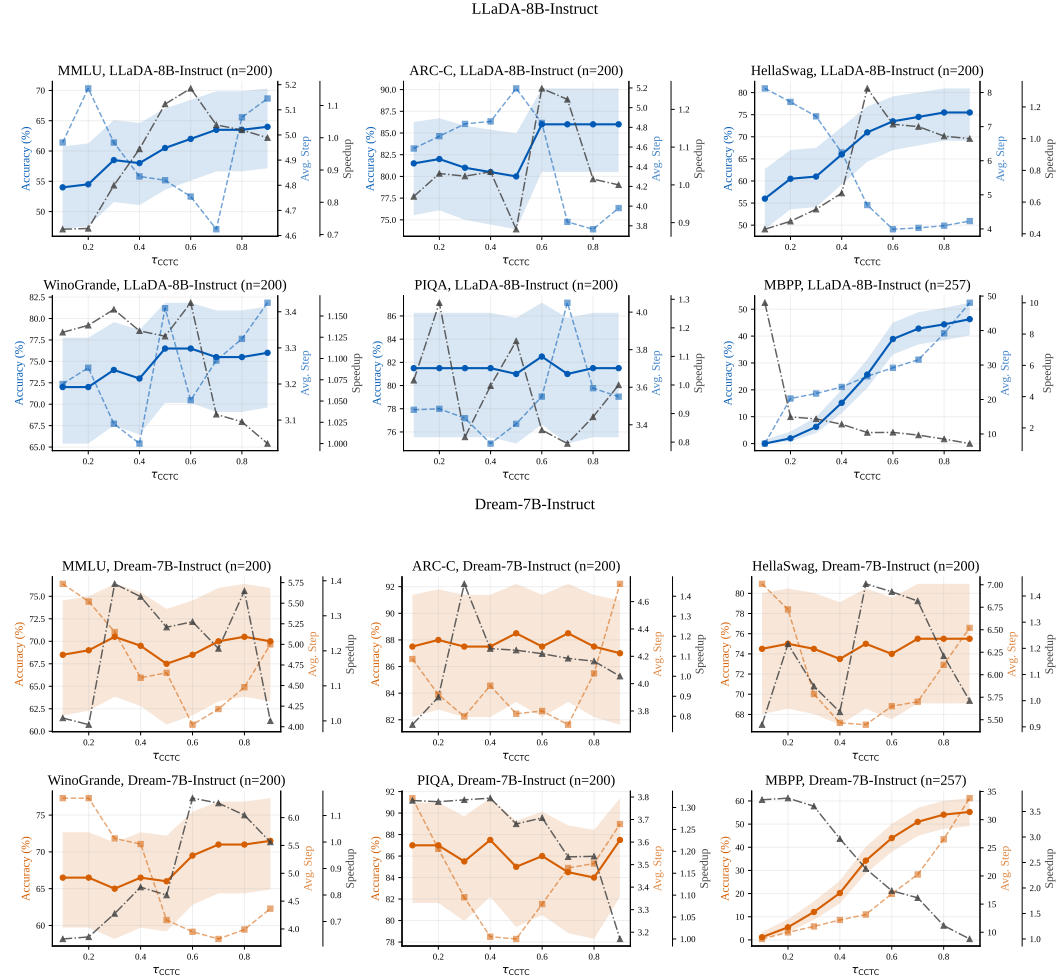


Figure 12: Effect of τ_{CCTC} on short-answer tasks and MBPP. LLaDA-8B-Instruct is shown above and Dream-7B-Instruct below. Solid curves show accuracy with Wilson-score 95% intervals, dashed curves show average Step, and dash-dot curves show Speedup relative to $\tau_{CCTC}=0.9$. Accuracy remains stable across most of the threshold range on the five short-answer tasks, whereas MBPP becomes less robust under more aggressive commitment. Measurements use an NVIDIA A100.

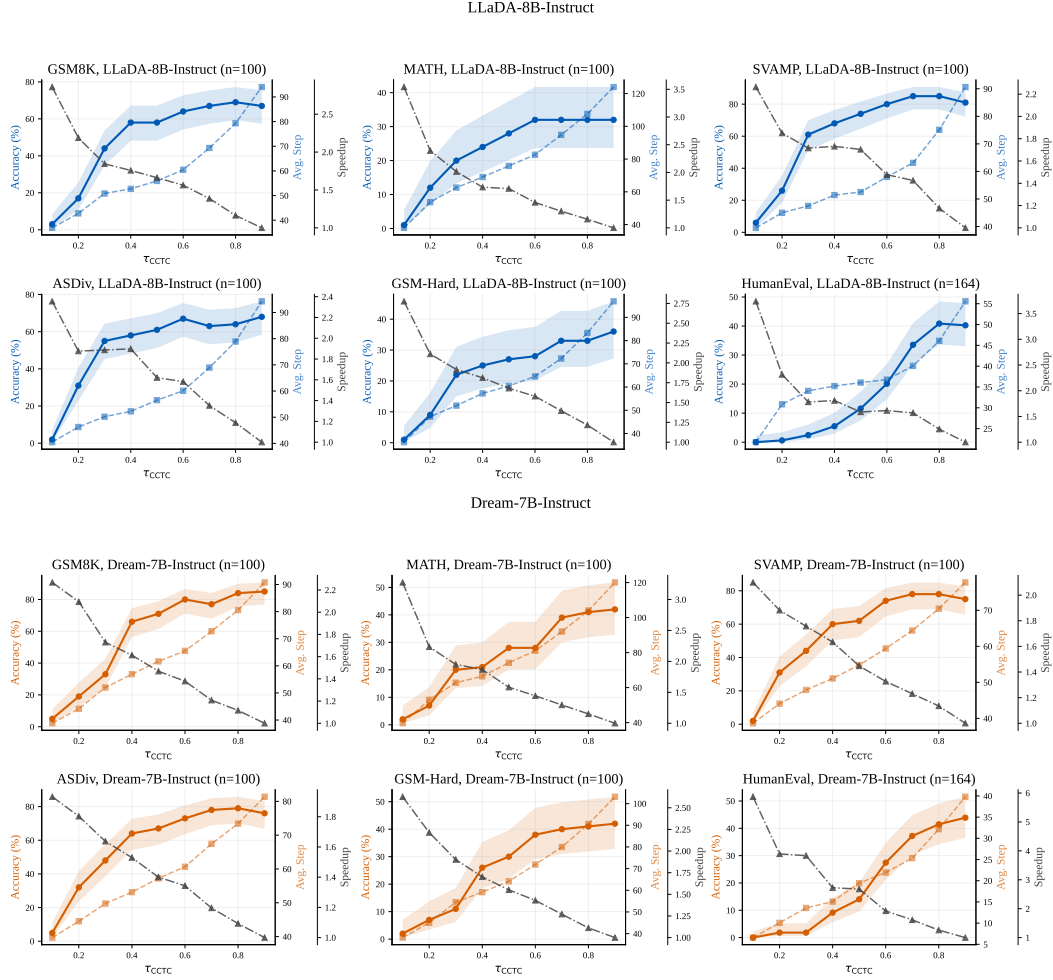


Figure 13: Effect of τ_{CCTC} on long-reasoning tasks and HumanEval. Accuracy decreases once the threshold permits sufficiently aggressive commitment, while Step and Speedup change more gradually. LLaDA-8B-Instruct is shown above and Dream-7B-Instruct below. Measurements use an NVIDIA A100.

G TEXT QUALITY AFTER EARLY EXIT

CVEE terminates once the extracted answer satisfies its confidence and stability conditions, after which the remaining masked positions are filled in one forward pass. Since the benchmarks in Table 1 grade only the extracted answer, this final fill can affect the surrounding generated text without changing task accuracy. We therefore measure its effect separately on the five short-answer tasks in Table 1 together with TruthfulQA.

Effect of the one-shot fill. We compare full-budget decoding with C^4 on the same evaluation examples and retain the complete generated text. Table 9 reports three quantities in addition to task accuracy. *PPL* is generative perplexity under a frozen GPT-2, *Changed* is the fraction of examples whose full generated text differs from full-budget decoding, and *Repeat* is the fraction of tokens identical to the immediately preceding token.

Task	LLaDA-8B-Instruct				Dream-7B-Instruct			
	Base PPL	C^4 PPL	Changed	Repeat	Base PPL	C^4 PPL	Changed	Repeat
MMLU	27.3	95.3	57%	19.5%	56.5	97.6	22%	7.0%
ARC-C	21.4	73.0	86%	26.9%	61.2	81.9	24%	12.6%
HellaSwag	44.6	65.1	35%	3.6%	48.4	100.5	49%	4.7%
PIQA	37.5	59.9	38%	2.1%	51.3	68.6	26%	1.4%
TruthfulQA	21.1	78.9	70%	7.6%	32.6	65.3	36%	3.4%
WinoGrande [†]	518.7	865.2	17%	—	1216.5	1198.6	2%	—

Table 9: Text quality outside the graded answer. PPL is generative perplexity under a frozen GPT-2. Changed is the fraction of examples whose full generated text differs from full-budget decoding, and Repeat is the fraction of tokens that repeat the immediately preceding token. The extracted answer remains unchanged on 98–100% of examples, while the surrounding text can degrade after the one-shot fill. [†]WinoGrande produces a median output of only two words, making repetition and perplexity less informative for this task.

The extracted answer is preserved on 98–100% of examples, and the five tasks shared with Table 1 reproduce the corresponding task accuracies. The surrounding text is more sensitive. For example, LLaDA reaches a repetition rate of 26.9% on ARC-C after early exit, compared with 0.0% under full-budget decoding. The effect varies substantially across tasks despite similar exit steps: ARC-C changes 86% of LLaDA outputs with 26.9% repetition, whereas PIQA changes 38% with only 2.1% repetition. This suggests that degradation depends more strongly on how much text remains unresolved when CVEE fires than on the exit step alone.

Effect of delaying the final fill. To test this interpretation, we optionally continue ordinary decoding for $\lceil \alpha \sqrt{t} \rceil$ additional steps after CVEE first fires at step t , and only then fill any remaining masked positions. This does not change the CVEE criterion; it trades additional decoding steps for less work in the final one-shot fill.

α	LLaDA-8B-Instruct						Dream-7B-Instruct					
	MMLU			ARC-C			MMLU			ARC-C		
	Acc	Step	Repeat	Acc	Step	Repeat	Acc	Step	Repeat	Acc	Step	Repeat
Baseline	64.0	64.0	0.0%	86.5	64.0	0.0%	71.5	64.0	0.0%	87.5	64.0	0.0%
0	64.0	4.8	19.5%	86.5	3.9	26.9%	70.0	4.5	7.0%	88.5	4.2	12.6%
1	63.5	6.9	18.9%	86.5	5.9	26.7%	70.0	6.4	7.8%	88.5	6.1	5.8%
2	63.5	8.4	15.5%	87.0	7.8	23.0%	70.5	7.2	5.6%	88.5	7.1	4.9%
4	63.5	9.9	12.6%	87.0	9.9	20.5%	70.5	7.6	5.0%	88.5	7.4	4.4%
8	63.5	12.7	7.9%	87.0	14.3	12.0%	70.5	8.3	2.4%	88.5	7.7	2.8%
16	63.5	17.8	1.7%	87.0	22.7	2.4%	70.5	9.4	0.2%	88.5	8.2	1.0%

Table 10: Effect of delaying the final fill. Increasing α gives ordinary decoding more steps after CVEE first fires and reduces the amount of text produced by the final one-shot fill. Repetition generally decreases as α increases, while accuracy remains within one example of the $\alpha=0$ result.

On both tasks, additional decoding substantially reduces repetition while leaving answer accuracy nearly unchanged. For LLaDA, repetition falls from 19.5% to 1.7% on MMLU and from 26.9% to 2.4% on ARC-C as α increases from 0 to 16. Dream requires fewer additional steps to reach similarly low repetition. The trade-off is therefore primarily between text quality and decoding cost rather than between text quality and answer accuracy.

We do not include this delayed-fill variant in the main results because the evaluated benchmarks consume the extracted answer rather than the full generated continuation. It nevertheless shows that much of the observed text degradation can be reduced by allowing more of the sequence to be resolved before the final fill.

H LIMITATIONS

- **The exit step degrades the text outside the answer.** CVEE fills every remaining masked position in one step, and that step writes worse text than ordinary decoding would. Appendix G measures it. On the short-answer tasks the fill inherits most of the buffer, and the text it produces repeats up to a quarter of its own tokens where full decoding repeats none, at generative perplexities two to four times the baseline’s. Accuracy is untouched because the extracted answer sits before the affected region, which is exactly why no number in Table 1 reflects it. The limitation is therefore one of scope rather than of correctness. C^4 as deployed suits a setting that consumes the extracted answer, and not one that consumes the whole generation. Exit momentum recovers most of the quality for step budget and is reported alongside, but it mitigates rather than removes the cause, since the fill still writes whatever remains in a single step.
- **Commits are monotonic.** All of C^4 ’s commits are also monotonic; once a position clears CCTC’s threshold or scheduled top- k quota, it is never revisited, even if a later step’s now-richer context would predict it differently. CVEE’s joint gate protects only the final answer span this way; an intermediate reasoning token committed under CCTC’s cheaper, non-final-block rule has no equivalent safety net. A mask-and-reconstruct verification pass would provide the missing check, asking whether the model’s own updated context reconstructs the same candidate before anything commits. It would also cost an extra forward pass per candidate, which competes directly with the speedup this paper targets. We leave that trade-off open, along with the question of whether the same cost is better reinvested in justifying a lower and more aggressive τ_{CCTC} .
- **Extending beyond a single tracked unit.** CVEE’s verification targets one identity-tracked unit per example, which on HumanEval is the single function each prompt asks for. Outputs assembled from several independent units fall outside that design, the natural cases being a multi-function program or a multi-hop response. Falling back to whole-buffer distributional stability would not extend CVEE’s logic to them, since it collapses into the same aggregate-confidence failure mode Section 4.1 identifies in Prophet. A more promising route decomposes the output into candidate units and applies CVEE’s identity-and-stability test per unit; CCTC’s final-block asymmetry would also need a different structural prior once no block is privileged this way. Pursuing this decomposition, and adding distribution-free risk control (Xie et al., 2026; Wynn et al., 2026) on top of the empirical tolerance used throughout this paper, are natural next steps.

USE OF LARGE LANGUAGE MODELS

We used large language models (LLMs) in three ways during this work, and all authors take full responsibility for the resulting content.

Research execution. LLMs assisted with implementing and debugging the decoding loop and its gates, the experiment and job scripts, the result-aggregation and table-generation code, and the figure-generation scripts. They were also used to audit reported numbers against the raw result files. Every LLM-written or LLM-modified piece of code was run and checked by the authors, and every number in the paper was traced back to the corresponding result file.

Drafting sections. LLMs produced first drafts of parts of the appendix and of several explanatory paragraphs, working from experimental results and outlines provided by the authors. The authors checked every drafted claim against the measurements and rewrote or removed text that the data did not support.

Writing assistance. LLMs were used to polish wording and grammar and to check the manuscript for internal consistency in terminology, notation, and cross-references.